

# 软件仓库管理 [03-Raw/26生成式软件工程/NJU]

---

原视频: <https://www.bilibili.com/video/BV1kybV6DE47> · 课程合集:  
<https://space.bilibili.com/202224425/lists/8942017> · 讲者: 蒋炎岩 (NJU 《生成式软件工程》2026)

## 全课主线

---

这节课的核心主张是: AI 能作为强大 assistant 帮人挡掉不顺心、大幅解锁执行力, 但前提是说清真实意图、训练细节注意力, 并理解软件工程 best practice。讲者想让大家在生成式软件工程中把项目组织成由 atomic change 推进、可回溯的 Git 快照, 同时让 agent 按 best practice 做事, 而不是只做被指派的一小步。

1. AI 可以挡掉不顺心, 但必须先交底真实意图, 并可蒸馏成 skills; GPT-6 能力提升且省 token, 却仍缺有经验工程师的 taste, 开放 intent space 会让它漂移。因此能力提升不等于替代有品位的工程师。(第 1 部分)
2. 讲者把 GPT-6 最大惊喜归为空间感知与理解, 并把 AGI 入口放在数学: 数学有全球共识、可做 grounding, 与软件工程的 intent specification gap 反向。因此数学和空间感知成为判断模型能力可被学出的两个支点。(第 1、2 部分) (整理者串联: 把“数学 grounding”与“空间感知”并列为两个支点)
3. 由此推论, 空间感知不是绝对学不会: 从简单空间任务可做 RL, 复杂空间想象也能做到; GPT-6 one-shot 已覆盖原先靠对话补足的应用。所以模型一旦学出能力, 会迅速覆盖旧工作流。(第 2 部分)
4. 模型同时具备空间感知与数学能力后, 在具身赛道明显能做好, 甚至未调 prompt 已达 SOTA。因此讲者判断模型一定会继续进步。(第 3 部分)
5. 但模型越强, 越要回答 AI 时代还要不要学数据库、操作系统等基础; 汽车史类比显示过去必须懂原理、今天无需懂也能开。讲者现阶段答案是肯定要学, 因为理解正确概念能更好解锁 AI 的创造力。(第 3、4 部分)

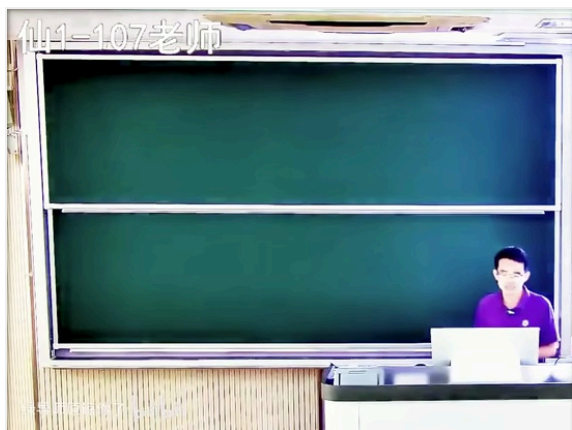
6. 进入生成式软件工程后，真实项目第一步是新建目录，项目被理解为“目录 + 文件”最终构成可交付产品；localhost 暴露了 intent 和 specification 的重要性，软件则是需求在信息世界中的投影。因此，管理这个目录及其历史成为核心问题。（第 4、5 部分）（整理者串联：从“项目是目录+文件”推出“需要管理目录历史”）
7. 于是需要版本管理：从 U 盘、压缩包、作业 V1/V2 到 Git；项目级时间回溯是刚需，因为人都会犯错；从 list/切换/回退可推出简化版 Git，分支、合并、冲突会在使用中自然出现，SVN 存 delta 不如 Git 追求快速回到任意版本。所以 Git 的根本出发点是项目级时间回溯。（第 5–12 部分）
8. Git 的 best practice 是改动一发生就 commit，message 要写清做了什么；不能把作业 V1/V2/V3 当成提交，空提交只在偶发触发流程时可接受，git add . 合法但不自动等于 best practice。因此 Git 难的不是命令，而是水面下的 best practices。（第 12–14 部分）
9. AI 可以审查提交、甚至重写历史，但 agent 时代更重要的是知道 Git 能干什么，并让 agent 按 best practice 做；git blame 追每一行来源，git bisect 把提交历史当数组定位 bug，每次提交应能通过构建和测试。因此 Git 的细节最终服务于可审计、可回溯。（第 15–17 部分）（整理者串联：把 blame/bisect/构建测试统一为可审计、可回溯）
10. 相应地，要训练学习意识和惊人注意力：中学“听话”策略会让人在 GitHub 上只做被指派的一步；多点点、多对比 README/路牌，virtually implement，先自己想再与外部材料比较，追问为什么别人更好。这样才能从 AI slop 和高密度外部材料中真正学到东西。（第 6–9 部分）
11. AI 让实现成本趋近于零，但仍会生成 slop；commit message 有 Conventional Commits 等规范，中英文空格等工程细节会形成一致性和可维护性，市场细节标准也会水涨船高。因此真正有价值的是愿意花时间学习并用 AI 消化概念。（第 24、25 部分）
12. 最后，项目管理本质是把人类处理不了的混沌拆成可理解、可审计的小任务：管理很多快照就可开分支尝试，失败丢弃、成功合并、并行推进；小 commit 让 bisect/blame/冲突局部化；架构阶段先用高智力模型定好接口、压小共享状态，再让多 agent 实现、人验收。因此 Git 是软件工程，核心是把项目组织成由 atomic change 推进的快照。（第 26–28 部分）

讲者的结论/号召是：在 AI 时代仍要学习基础、训练细节注意力，并让 Git/agent 按 best practice 把项目推进成一串可回溯的 atomic change。

## 第 1 部分



快讯：GPT-6 发布了



但设计上还是要骂

我好像还可以再苟活几个月

- 当然这是因为面对了非常开放的 intent-spec-impl 空间
- ♦ “做一个好用的界面”：什么叫好用，往往看到第一版才说得清。
  - ♦ 实现会反过来改变需求：设计过程中，题目本身也在变。

开放的问题，也能逐步变得可求解

- ♦ Simon 在 1973 年指出：结构良好与结构不良的问题之间，边界是流动的。(The Structure of Ill Structured Problems)
- ♦ 由此看设计：每一次反馈，都在把“我想要”变成更明确的约束：今天还要骂，并不意味着这会是永久的能力
- 边界。



“理应当做到”的事就是能做到

我对 OpenAI AGI Roadmap 的理解：一切对齐到数学逻辑推理

- 直接杀死了比赛 😂 (具身赛道的人应该在瑟瑟发抖)



做了小半年的工作直接👏了  
Baseline 后设置不可能比得过了。心理 PhD  
Student 一分钟😭😭😭 #我要工作工作工作 #一想  
就上班班班班  
© 地点 泰康之家养老社区 另图 15km

## AI 是帮人挡掉“不顺心”的 assistant，前提是说出真实意图

- 前几次课讲怎么用 AI 的案例；刚才讲者现场教学，示范如何把 AI 整合进自己的生活流程。
- 生活中会遇到无穷多不顺心，用很多工具时都会感到不顺心。AI 作为非常强大的 assistant，可以帮你把不顺心的事情挡掉。

- 你只需要告诉它你的**真实意图**。
- 还可以把这个过程**蒸馏成 skills**，让流程更平顺。
- （整理者串联）这段的链条：遇到不顺心 → 交底真实意图 → AI 代理/挡掉 → 蒸馏成 skills → 生活流程更平顺。

## GPT-6 发布：惊喜在能力提升，缺口在“品位”

---

- GPT-6（转录作 GP6）真的发布了；讲者用过，认为还是有惊喜。
- 讲者认为编程上的提升有限：有经验的软件工程师有**好的品位/taste**，这种品位可以指导 AI 做出更好的东西；但 GPT-6 更多是在**完成任务**，还没有学会很好的品位。讲者常骂它，并开玩笑说自己可能还能再“苟活”几个月。
- 不过 GPT-6 各方面能力都提升了；做更复杂任务时更快，**省很多 token**。讲者特意 measure 过，确认如此。
- （整理者串联）所以：能力提升 ≠ 替代有品位的工程师。

## 觉得 GPT 做得不好，常因开放的 intent space 无法直接与人脑对齐

---

- 讲者指出，我们面对的是一个非常开放的**意图空间 (intent space)**。
- 同一个任务，例如“给我做一个国民级的应用”，每个顶级产品经理脑袋里想的都不一样。
- AI 没有办法和人脑直接做这样的对齐，所以一定会按自己的方式**漂移**。
- 因此觉得 GPT 做得不好，往往是因为开放意图空间下的对齐问题。

## AI 代码越来越紧凑，人类越来越不 review

---

- 讲者发现 AI 写出来的代码，人类越来越难读。
- GPT-6 可能受**思维链变短**影响：思维链变得非常紧凑，写出的代码也非常紧凑。
- 从 GPT-5 到 GPT-5.6，再到 GPT-6，代码写得越来越紧凑，人越来越不想看，没办法 review。
- 曾经会用各种 prompt 让 AI 写出符合人类工程规范的代码；现在讲者认为我们再也不看 AI 写的代码了，随它去，反正它能**把代码弄对**。
- （整理者串联）因果链：思维链压缩 → 代码紧凑 → review 成本升高 → 人类放弃审阅、只看结果正确。

## GPT-6 最大惊喜：空间感知与理解；讲者把 AGI 的入口放在数学

- 讲者认为 GPT-6 带来的最大惊喜，是它的空间感知和理解能力。
- 讲者一直认为，OpenAI 对 AGI 的理解是：AI 首先得学会数学。
- 原因是数学是所有人类世界里面最难的那种语言；它有一个全世界所有人都认同的推理系统。
- 这与软件工程中的 intent specification gap (ASR: intent spec in cation) 相反：数学有全球性的共识。

## 第 2 部分

仙1-107老师



给老师您递个了 bilibili

### “理应当做到”的事就是能做到

我对 OpenAI AGI Roadmap 的理解：一切对齐到数学逻辑推理

- 直接杀死了比赛 😂 (具身赛道的人应该在瑟瑟发抖)



做了小半年的工作直接 😂 了  
Baseline 应该是不可能比得过了。心得 PhD  
Student 一分钟 🧐 #我要工作 我要 #一想  
别上班就躺平不  
📍 地点 香港之家养老社区 另图 1.8km

3 / 3

仙1-107老师



给老师您递个了 bilibili

### 但设计上还是要骂

我好像还可以再苟活几个月

- 当然这是因为面对了非常开放的 intent-spec-impl 空间
- ✦ “做一个好用的界面”：什么叫好用，往往看到第一版才说得清。
  - ✦ 实现会反过来改变需求：设计过程中，题目本身也在变。

开放的问题，也能逐步变得可求解

- ✦ Simon 在 1973 年指出：结构良好与结构不良的问题之间，边界是流动的。(The Structure of Ill
- Structured Problems)
- ✦ 由此看设计：每一次反馈，都在把“我想要”变成更明确的约束：今天还要骂，并不意味着这会是永久的能力
- 边界。

2 / 3

## 数学与软件工程的 intent specification gap 反向：定理有全球共识，可做 grounding

- 讲者认为，数学与软件工程里的 intent specification gap 是反着走的。

- 数学有全球性共识：一个定理只要能证明出来，全球的人都会判断这个证明对或不对。
- 因此，讲者认为数学是非常好的、能 **grounding** 的东西。
- 讲者进一步主张：数学的逻辑推理可以解释人类生活里的各种行为。

### 逻辑链：

1. 数学证明有全球共识 → 定理可被全球判定对/错。
2. 因此数学是很好的 grounding 对象。
3. 讲者认为数学逻辑还能解释人类行为 → 可把日常决策也接到数学/逻辑上。

## 午饭选择不是“灵魂”决定，而是多因素投票，可落到命题逻辑

---

- 例子：今天中午吃什么、去挤食堂，是马上会面临的问题。
- 你可能觉得这是人类世界的问题，必须由“灵魂”给出答案；讲者说不完全。
- 讲者说，大脑其实在不停“投票”：他自己会选择在义父楼（保留转录；疑为“逸夫楼”）订饭，因为对他来说这是最快的选项；他下午还要开会；如果时间来得及，中午还可以小睡一会儿。
- 所以，讲者认为自己是在各种因素之间投票。
- 这些因素之间的关系，比如  $(A \rightarrow B)$ ——“如果我做了 A 就会做 B”——可以 grounding 到离散数学里的命题逻辑（转录为“立命题逻辑”）。
- 讲者据此推测 OpenAI 的路线：先把数学学好；现在数学已经学会了，然后再用强化学习去学别的能力。

## 空间感知不是绝对学不会：从鲁班锁、骰子展开图等简单任务开始 RL

---

- 讲者认为，空间感知不是那种绝对学不会的能力。
- 例子：给一个很复杂的 鲁班锁，问按什么方向可以把第一块抽出来，人脑就会开始推演。
- 小时候的智力题也类似：给一个立方体或奇怪形状的投影，问哪一块和它一样；或者给一个六面骰子，问哪一个展开图对应，人会在脑袋里做空间想象。
- 有些 benchmark 里就是这样的题。
- 讲者判断：只要从简单的空间任务开始给模型做强化学习，复杂空间想象肯定也能做到。

逻辑链：简单空间任务可训练 → 逐步扩展空间想象 → 空间感知能力可被学出来（这是讲者的明确断言）。

## 3D 打印与光栅玩具：从对话辅助到 GPT-6 one-shot

- 讲者看到相关例子时，GPT-6（转录为 GP6）还没有发布。
- 他心疼自己的 PhD student：他们已经做了小半年的 3D 打印工作。
- 在 GP5.6（保留转录；按上下文指某代 GPT）空间感知能力很差的情况下，他们仍能通过直接对话生成一个还像样、各方面符合规范的 3D 打印模型。
- 他们也有一些比较 killer 的应用：做小的 光栅玩具（转录为“光山玩具”；他也说“光栅尺”），从左边看和从右边看是两个不同图案。
- 比如可以说“我要做这个”，再用一个很长的搜索树帮你做一个光栅纸/光栅卡（转录为“光产纸”）。
- 结果昨天 GPT-6 one-shot 直接出，直接搞定。
- 讲者说，他们现在还在想怎么解决这个问题；他把这一变化同样归因于模型已经具备了空间感知能力。

小结：讲者用数学 grounding 与空间感知两个例子支撑同一判断：模型能力可以被逐步学出，且一旦学出，会迅速覆盖原先靠人对话补足的应用。

## 第 3 部分

仙1-107老师

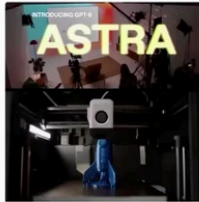


账号 你爱谁你了 bilibili

“理应当做到”的事就是能做到

我对 OpenAI AGI Roadmap 的理解：一切对齐到数学逻辑推理

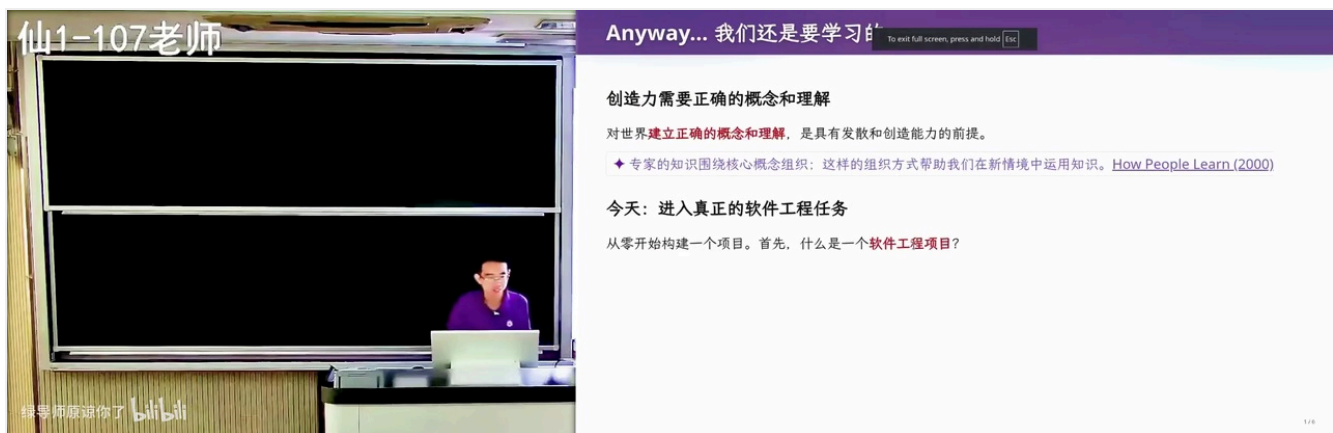
- 直接杀死了比赛 🤖 (真身赛道的人应该在瑟瑟发抖)



做了小半年的工作直接🔥了  
Baseline 应该是不可能比得过了。心疼 PhD Student 一分钟🥺🥺 #我爱工作工作爱我 #一想就上把眼睡不着

📍 地点 泰康之家养老社区·吴郡 1.8km

3 / 3



## 模型有了空间感知与数学能力后，在具身赛道已能做得很好

- 讲者仍在想如何解决前面的问题；接着指出：当模型有了空间感知能力，又有数学能力之后，它在具身赛道明显可以做得很好。
- 证据：第二天小红书上很多人发，直接用 GPT6 控制机械臂，完成抓取、叠衣服等复杂动作。
- 讲者提醒：GPT6 本质是语言模型；虽然它有视觉能力，但只是“一帧一帧地”问下一步要做什么，而且还没有用上 prompt engineering（提示词工程）。
- 即便如此，讲者认为它已经做得相当好，达到 state of the art / SOTA。
- 逻辑链：空间感知 + 数学能力 → 具身任务可做好 → 语言模型可控制机械臂完成抓取/叠衣 → 未调 prompt 已达 SOTA → 模型一定会进步。
- 讲者半开玩笑：也许某次上课时，他突然就可以说“这课不用上了”，大家就可以直接去做产品。

## AI 时代还要不要学数据库、操作系统？讲者用汽车史作类比

---

- 讲者认为今天仍需要学一些**软件工程**知识；并重申：**AI 解锁的是人的执行力**。
- 有人问讲者：AI 时代为什么还要学、还要不要学**数据库、操作系统**这些。讲者说自己答案也不是特别确定，于是打了汽车比方。
- **100 多年前汽车刚发明时：**
  1. 汽车开 **12 km** 就会出故障。
  2. 所以每个司机/driver 都必须非常清楚汽车工作原理。
  3. 因为车一旦出故障，就得知道它是怎么工作的。
  4. 那个时代，想开车就要掌握汽车基本原理，这是理所应当的。
  5. 类比到 **computer science / 软件工程**：就要学**操作系统、数据库**，把基础打好。
- **今天开车：**
  1. 在地下车库直接说“帮我开到什么地方”，**自动驾驶汽车**就出发。
  2. 人已经不理解整个系统是怎么工作的。
  3. 但讲者指出，最终动力还是传递到一个机械装置，和 100 多年前的汽车仍有一点相似。
  4. 中间经历的所有东西，你都不需要知道，就可以把汽车开起来。
- **类比结论：**人工智能可以想象成类似汽车的东西，而且比汽车更强；它替代的不只是人的腿部向前走的能力，还替代人的**智力**。于是问题变成：那我们是不是还要学？
- （整理者串联）这个类比的关键是：汽车从“必须懂原理”走到“无需懂原理也能开”，对应到软件基础是否还要学的问题。

## 讲者答案：现阶段要学，但未来怎么学不知道

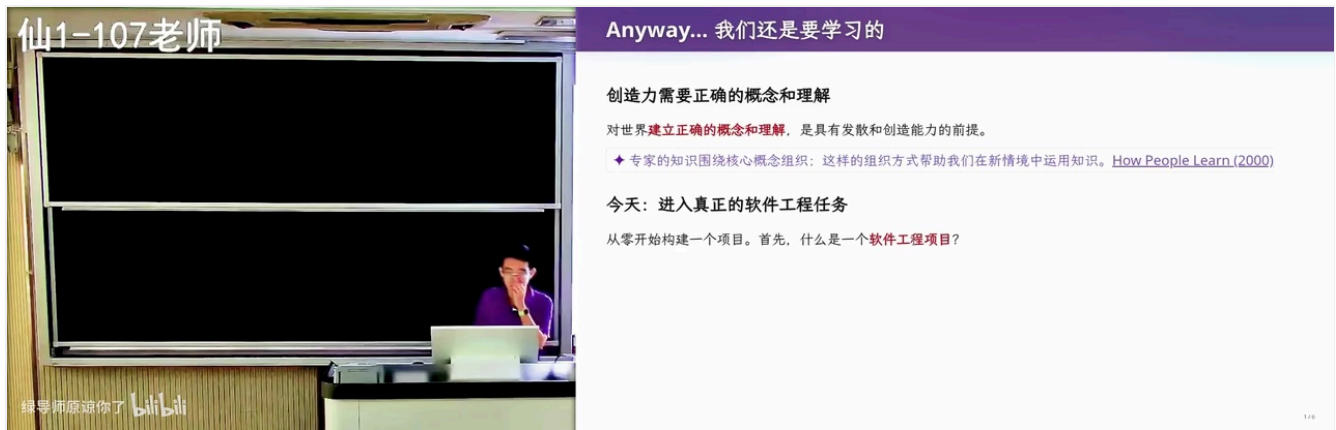
---

- 讲者今天的答案：**肯定要学**。
- 理由：你们学了以后，可以更好地**解锁 AI 的创造力**。
- 但“以后应该怎么学”，讲者暂时不知道。
- 讲者欢迎学生如果有答案来探讨；并强调这是**个人观点**。
- 讲者认为，至少在现在的阶段，学习还是有好处的。

小结：讲者主张现阶段仍要学基础，因为学习能更好解锁 AI 的创造力；未来学习方式则尚未确定。

---

## 第 4 部分



### 学习正确概念仍有益：它能帮你从别处借鉴知识，创造更好的东西

- 讲者先说明：如果你们有答案，也欢迎来和他探讨；以下是他个人观点。
- 讲者认为：至少在现在阶段，学习仍然有好处。理由不是死记，而是理解正确概念后，可以帮助你从别的地方借鉴知识，创造更好的东西。

### 进入生成式软件工程：第一步不是下载作业，而是新建目录

- 讲者今天进入“真正的软件工程”，即生成式软件工程。
- 如果开始做软件工程 / vibe coding（原文转写为“web coding”，按语境疑似）：做作业时，你通常从课程网站、老师要求或某个下载物开始；但真正想做项目时，讲者说第一件事一定是建一个目录，给它起名字。名字甚至可以是中文，比如“我的最酷的game”。然后才开始 vibe coding。

做作业：课程网站/老师要求 → 下载一个东西 → 从那里开始工作。

真做项目：新建目录 → 命名 → 在目录里开始 vibe coding。

（整理者串联）这两条路径的差别在于：作业从已有骨架开始，真实项目要先建立自己的项目容器。

### 软件工程项目是什么：一个目录 + 一堆文件 = 可交付产品

- 讲者问：创建后面对一个空目录，你们怎么知道该做什么？

- 他回顾学习方式：看老师上课打开一个工程项目，里面有文件，于是无意中记住一些东西；老师或同学推荐某个发布物，你拿到 **GitHub** 或 **git** 链接，打开网页，看到人类友好的界面上有一些文件，于是知道“我的目录里会要有文件”。
- 所以开始一个软件工程项目时，你会建一个这样的目录，里面有一堆文件。
- 这些文件共同构成一个可以交付的东西，最后会变成你的产品。
- 这个目录可能直接在本地运行：你对 **agent** 说“请你帮我运行”，窗口或网页就弹出来。

看老师项目 / 打开 GitHub 链接 → 看到文件结构 → 记住“目录里要有文件” → 建目录并放文件 → 得到可交付产品

## 本地能跑 ≠ 世界能访问：localhost 暴露了 intent 和 specification 的重要性

---

- 讲者讲一个著名笑话：**GPT-6**（原文“GP6”）太牛逼了，我又做了多少个 **killer 级产品**，它们是 `http://localhost:5173`（原文转写为 5137）下的东西，然后发个 Twitter 等。
- 但讲者说：你还是需要支持。你在本机可以打开它，世界上其他人打不开它，因为 **localhost** 是你的本机。
- 可以把 **intent** 和 **specification** 说得更清楚。比如在 **intent** 文档里说明“我的这个东西要被全世界人看到”。
- 如果你不说，**GPT** 可能把你当成一个开发者。

讲者给出的链条：

1. 不说明“要被全世界看到”
2. → **GPT** 按开发者默认方式处理
3. → 开发者先本地调试，起一个本地 **development server**
4. → 你可以看到结果，直到觉得 **ready**
5. → 才把东西送到远端

## 这些仍是计算机科学知识：课程反复教，最终服务于可交付物

---

- 讲者说，这都是知识。你们是 **computer science** 学生，上各种各样的课时反复学习这些知识。
  - 这些知识最后变成：目录里有文件，最后得到一个可交付的东西。
-

## 第 5 部分

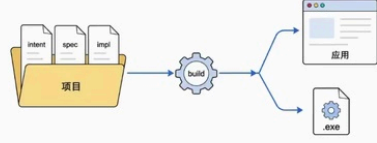
仙1-107老师



### 什么是一个软件工程项目？

一个目录，一组文件，一个可交付的东西

- 目录里的文件，混合了 intent、spec、impl、.....
- 经过 build / 运行，可以得到 npm run dev 的本地窗口，也可以是一个 .exe。



2 / 6

## 软件工程项目从“能生成 binary”开始：一键编译 → 命令行工具 → GCC

- 讲者说，上各种课、反复学习这些知识，最后会得到一个可交付的东西。
- 学编程的起点：可能是在 WHCV（原词保留）里一键编译和运行。
- 后来你们会知道：**build** 目录里所有的东西，都是由某些**命令行工具**创建出来的。
- 到大二，你们会用 **GCC** 这样的命令行工具，把一个 **hello.c** 变成 **hello binary**。
- 讲者的结论是：这就构成了一个**软件工程项目**。
- 逻辑链：上各种课/反复学知识 → 得到可交付物；一键编译运行 → 理解 build 产物由命令行工具生成 → 用 GCC 把 hello.c 编译成 binary → 软件工程项目。

## “什么是软件”：标准定义是代码+文档+相关集合，讲者定义为需求在信息世界的投影

- 讲者提到，以前老先生喜欢问：“什么是软件？”
- 他先强调软件的基础性和无处不在：
  - 没有软件，今天看到的是黑的；
  - 不光是看不到讲者本人，连这个台子也启动不了；
  - 他的升降桌也升不起来、降不下来；
  - 升降桌这边还有高度显示，现在是 **106 厘米**。
- 有一个**标准定义**：软件是代码、文档等等，一切和软件相关的东西的**集合**。
- 但讲者说他一直讲：**软件是人类世界的需求在信息世界里的一个投影**。

- 这个投影今天看到的是一个目录，目录里面有各种各样的东西。

## 有 hello.c 并编译出 hello，就已经在做软件；只是不一定符合现代 best practice

---

- 讲者说，你知道目录里面应该有什么。比如有一个 `hello.c`，然后你可以编译出一个 `hello`。
- 讲者认为，这时你已经在做软件了。
- 当然，这可能不符合现代软件工程的 `best practice`。
- 讲者认为古早时期，甚至 10 年以前，大家真的是这样做的。

## 第一个 PhD 学生的 U 盘故事：方便保存代码，但一次损坏就丢掉作业与启蒙作品

---

- 讲者讲他第一个 PhD 学生的故事。这个学生应该是 2012 年入学南大；是讲者的第一个博士。
  - 学生大一发现了一个“非常好的学习方式”：经常去机房。
    - 原因：自己的笔记本电脑屏幕比较小，也不很舒服；
    - 不像现在，大家应该都会买一块屏幕，不贵，甚至淘一个二手屏幕只要小几百块钱，就能在宿舍获得非常好的编程体验；
    - 但那时好像标准答案是去机房，屏幕没那么便宜。
  - 去机房要来回拷贝代码，比较麻烦。
  - 他大一想出的妙招：把代码保存到 U 盘上。讲者说，你们多多少少可能也干过，或者在中学电脑课上把作品保存在 U 盘上。
  - 结果到他大二的时候，这个 U 盘坏了。
  - U 盘坏了以后，代码全部没了，作业不见了，他珍藏的启蒙时期的作业也不见了。
  - 讲者说他有一点伤心，但好像也没有太大的关系。
-

## 第 6 部分



### 从丢失的启蒙作业说起：旧式代码管理留下的遗憾

- 讲者提到，珍藏的启蒙时期作业不见了，有点伤心；但他又说似乎也没有太大关系。
- 他借此回忆：我们很长时间都是这样管理代码的——上学时交作业是打一个压缩包发邮件；现在学生好像是在网页上上传。讲者认为，这些都是管理项目的方式。

### 软件工程今天有标准答案：应该用 Git

- 当谈软件工程时，今天有标准答案：你应该用 Git。
- Git 除了能保存一个目录，还能保留、比较和恢复历史版本。项目本身是一个目录，里面有 `hello.c`、`hello` 的 binary、文档等。
- 讲者认为“保存/恢复历史版本”是很重要的软件工程特性，后面会详细展开。
- 逻辑链：项目=目录+文件 → Git 额外保存版本历史 → 可以比较、恢复 → 成为软件工程的标准工具。

### Git 让作业分发和项目管理从“邮件+压缩包”变成“远端同步+自动提交”

- 有 Git 后，很多事变得容易。讲者说自己再也不用发邮件了。
- 他对比今天：今天是 AI 的时代，有十倍、百倍的生产力提升。他只要让 AI 帮写一个 `commit`（原文：投诉 / Complain），就什么也不用管；如果用 Codex，他会选择信任它，甚至看都不看，结束后让它把浏览器关掉。而过去真的还在用邮件管理项目。
- Git 提高效率的具体方式：
  - 用 `git clone` 从远端获取作业。

- 作业更新时，讲者可以在远端更新，不需要给每个同学都发压缩包；只需上课告诉同学们“我的作业更新了，你们用 `git pull` 下载”。
- 讲者上课有一个 `make submit` [原文识别为“mix submit”]，会自动用脚本把包打好，再上传到课程网站，非常方便。
- 讲者认为这有点像现代 CI/CD 流程；他说 CI/CD 应该会专门用一次课来讲。

## 学习变容易后，这门课的知识不重要，重要的是学习意识

- 讲者说这引出他几乎每节课都会讲的事情：当今天学习变容易以后，这门课上讲的知识都不重要，重要的是你们的学习意识。
- 他举例：你们肯定看过很多 Git 项目，因为打开课程作业就会看到 GitHub 链接。
- 但要留意不同项目之间的细微区别：有的项目有这个文件，有的项目没有那个文件；有的项目文档这样写，有的项目文档那样写。
- 讲者认为，在这种不断有信号进入的时候，你其实就能判断出我们应该怎么用 Git。
- 最后他提出一个问题：是不是总有一种感觉——“老师，老师说的事情实在是太多了”？这是不是跟中学有关？因为在中学时你必须听老师的话。

小结：Git 把项目从“文件包”变成可追溯的历史；工具让学习变容易后，讲者把重点转向观察信号、形成判断的学习意识。

## 第 7 部分

仙1-107老师



指导老师原谅你了 bilibili

### 学习变容易以后，学习意识更重要

大量别人的项目，就是你的学习素材

- 公众号上满天都是 GitHub 链接。
- 哪怕没有人告诉你哪个好、哪个不好，看得多了，也会有感觉。

“上课耽误学习”的本质

课外的信息量、信息密度巨大，但我们从未教过**如何用好**。

◆ 把一次浏览变成一个具体问题：这个项目里的哪一个细节，是我自己做时不会想到的？

5/6

## 中学“听话”策略带到 GitHub，会退化成只做被指派任务的 AI

---

- 讲者把问题追溯到中学：中学时老师要说的事情太多，学生必须听老师的话，才能把所有事情做完；不听老师的话，就做不完。
- 到了现在，比如在 GitHub 上学习时，如果还带着这个心态：老师说“好，你去把作业拉下来”，你就只把作业拉下来，**绝对不会多碰一点点**。
- 讲者给出的理由：因为这是**最省时间**的做法。
- 他的类比：这就像今天干活的 AI，被强化学习调成了“当老师说让你做这件事的时候，你只做这件事”。
- 讲者因此劝大家：愿意**多花一点点时间**点进去，看看里面的各种东西。

链条可写成：

1. 中学任务多 → 必须听话 → 形成“只做被要求的事”的省时策略。
2. 到 GitHub 仍这样 → 只完成“把作业拉下来” → 像被强化学习成只执行单一指令的 AI。
3. 讲者主张：多花一点时间点进去看，才可能得到额外收获。

## 多点点、多对比，才能从 AI slop README 中认出好文档

---

- 讲者说，多花一点时间，你可能突然灵光一现：“这个文档写得很好嘛。”
- 具体例子：今天满世界 GitHub 上的项目，全部都是巨长无比的 README；README 里有各种 Unicode 字符组成的目录、结构框图；讲者称这种为 AI slop 的感觉。
- 看多了这种 AI slop 项目后，突然发现一个几十年前开始维护的老项目：
  - 点进去，文档清新舒服；
  - README 很短；
  - README 看完后，你想看的东西就是三个链接，点过去就行；
  - 不需要读那么长的 AI slop。
- 讲者由此说：你就学会了文档应该这样写。
- 他还问：你们有没有留意过这些细节？
- 逻辑链：
  1. 只做“把作业拉下来” → 不会点进去看细节。
  2. 多花时间点进去 → 看到大量 AI slop README，也看到简洁老项目 README。
  3. 两者对比 → 突然学到好文档的标准：短、直接、给关键链接。

- （整理者串联：这里的“对比产生学习”，是讲者后面用路牌例子继续说明的同一机制。）

## 路牌对比：好设计是在你不确定时，答案就在正前方

---

- 讲者说，他不确定大家有没有机会去对比不同城市的路牌；他觉得这很有意思。
- 他提出：为什么我们有时觉得中国是一个 **developing country / 发展中国家**？讲者认为这意味着**提升空间很大**。
- 香港经历，好多年前，他特别 **get impressed**：
  - 他在香港找路；
  - 每当有不确定性，比如前面有三个口，正前方一定有一个路牌，告诉他三个口是什么；
  - 他形容“从来没有过”（上下文指向：不需要像在南京那样到处找不到指示）；
  - 那里有一个非常复杂的**地下系统**。
- 南京对比：
  - 回到南京，有时经常要在地铁站里找下一步；
  - 不知道以后要到哪一个口；
  - 需要环顾四周，甚至移动比较远的距离，找到一个地图再看。
- 讲者把这个对比称为一个**学习过程**：如果有一个范例比你做得好，你就学到了。

## 从路牌迁移到 UI/README：站在人的角度，用最少方式呈现 A/B/C

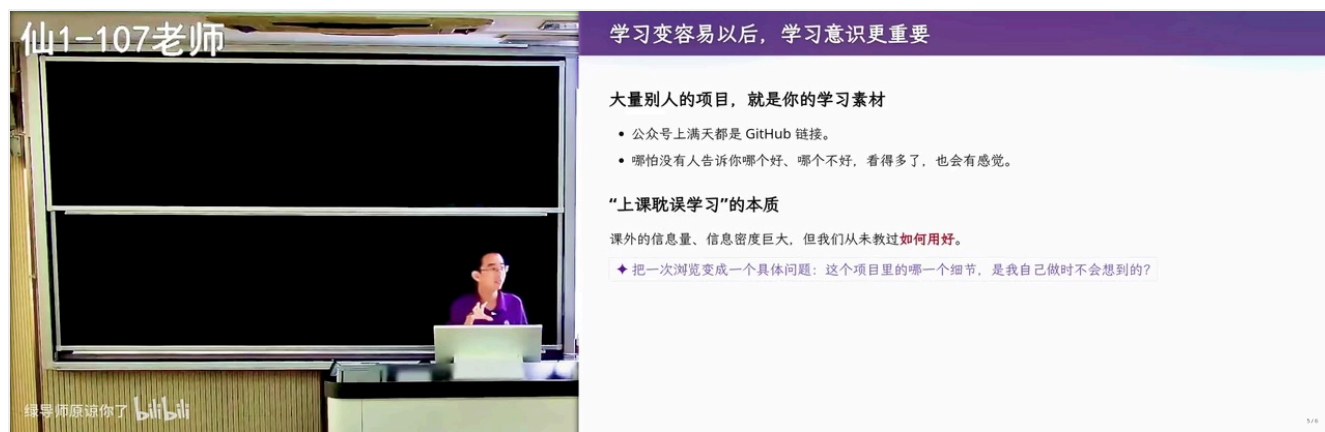
---

- 讲者说，从这个对比中他学到一个 **lesson**：
  - 如果他要做 **UI 设计**，UI 应该让任何人站在页面上时，先想：这个人会想什么？
  - 不同的人，他要的 **A、B、C** 会趋向不同方向；
  - 他希望用**最少的方式**把 A、B、C 全部呈现出来。
- 最后他用一句话收束：所以你看，这就是一个 **README**；讲者说“你们都会用 AI 生成 slop README”。
- （整理者串联：把路牌、UI 和 README 连起来，讲者的标准是：读者在不确定时，应该立刻看到下一步该点哪里，而不是被冗长装饰淹没。）

小结：讲者用 GitHub README 与城市路牌说明：别只做被指派的一步；多看、多对比，再站在使用者角度用最少信息给出下一步。

---

## 第 8 部分



### README 的目标：用最少数方式把不同人需要的 ABC 全呈现出来

- 讲者先把人区分开：不同的人，他要的 **ABC** 会趋向不同方向。
- 讲者的目标是：用最少数的方式把 **ABC 全部呈现出来**。
- 所以他把 **README** 当作例子：**README** 就是这种“最小但全”的呈现载体。
- 讲者说：你们都会用 AI 生成 **slop README**（原文识别为“slap read me”）；但看过几个好的 **README** 后，你就会写 **README**。
- 有了 AI 后，可以马上问它：这个 **README** 为什么比那个好？为什么这个项目的 **README** 没有 **AI slop** 的感觉（原文作“AI stop”）？AI 会告诉你。

### 学生抱怨“上课耽误学习”的原因：课外信息密度远超课堂

- 讲者说，他因此理解为什么很多同学怨气很大，觉得上课耽误学习。
- 他的理由：现在课外的信息量和信息密度，已经远远超过任何一个课堂能做的事情。
- 具体例子：打开一个成熟的 **GitHub 项目**，那是这个工程师十几年的心血；甚至 10 年前这个项目崩溃过一次，他做了一次大版本重构，发现以前的 **practice** 不够好，最后你才看到现在这个好好的东西。
- 逻辑链：
  1. 成熟项目承载长期试错、重构后的高密度经验。
  2. 课堂能提供的信息密度难以匹敌这种外部材料。
  3. 所以学生会觉得上课耽误学习。
- 但讲者补充：传统教学方法没有教大家怎么把这些海量、高密度信息用好；大家也没有准备好去拥抱这样的知识。

### 讲者认为自己当老师的原因与要训练的能力：惊人注意力

- 讲者认为，他现在做老师，是因为他有这种 **惊人的注意力**。
- 他认为大家要训练的也正是这种惊人的注意力。
- 这种注意力表现为：
  - 看到任何东西之前，大脑先把眼前看到的所有东西忽略掉。
  - 先想：如果我要自己来解决这个问题，我会怎么做？
  - 然后立即比较：我看到的和我脑袋里所想的東西，谁更好？
  - 如果我做得更好，就认为这是一个 **slop**（原文作“slob”，按上下文指低质作品）。
  - 如果发现他做得比我好，就很惊喜：诶，他为什么做得比我好？
- 讲者认为，养成这个习惯后非常“恐怖”：你在任何时候都在疯狂地从外面接收信息；你能留意生活里各种小细节时，就不停地在提升。

## 读论文/学课程的方法：virtually implement

- 讲者说，很早大家就有推荐：如果你们要读一个 **research paper**，研究论文应该怎么读？
- 他提到早先展示过：AI 会被 **注意力机制** 带到作者的 **逻辑陷阱** 里面去。
- 他给出的好学习方法叫 **virtually implement**（在脑中先实现一遍/虚拟实现），也适用于学任何课程，包括 **高等数学、计算机系统基础**。
- 做法：
  1. 你现在有一个具体的问题或具体的需求。
  2. 先在脑袋里把这个问题自己做一遍。
  3. 你会得到一个什么样的结果？
  4. 如果你的方法和作者的方法一致，那很好，说明你已经学会、完全掌握这个知识。
  5. 如果你不一致，讲者说还有两种情况；这一段只说到第一种：
    - 成熟、好的教科书上的方案，比你想象的方案更好、更精巧、更简洁。
    - 这时你就要想：为什么他能想到？

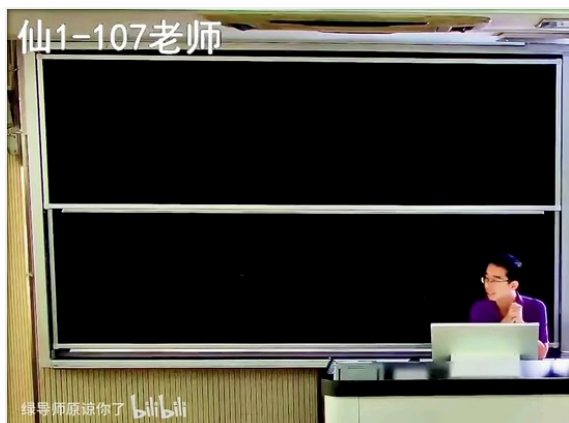
## 逻辑链总览

- 讲者给出的链：
  - 外部材料（README / GitHub / 论文 / 教材）信息密度高 → 传统课堂无法匹配 → 学生会觉得上课耽误学习。
  - 要利用这些材料 → 先自己想一遍 → 与材料比较 → 判断谁更好 → 若对方更好，追问“为什么他能想到”。
  - 学习上的具体化 → **virtually implement** → 一致=掌握；不一致=进入追问。

- （整理者串联）本段把 README、GitHub、论文阅读和课程学习串到同一件事上：不是被动接收，而是先形成自己的方案，再拿外部方案做对照，用差距驱动理解。

小结 讲者用 README 和成熟 GitHub 项目说明：外部材料的信息密度已超过课堂，真正要训练的是“先自己想、再比较、再追问为什么”的惊人注意力；在论文和课程学习中，这个方法叫 virtually implement。

## 第 9 部分



留意：让每一次阅读都成为学习

**Virtually Implement：先在脑中做一遍**

- 看到问题，先问自己：我会怎么想？如何 approach？
- 再看作者的做法：
- 如果一致——很好。
- 如果不一致——就是一个学习的机会。

**对细节保持注意力**

- 读别人的东西，甚至 AI 生成的东西，都要有“感知细节”的意识。
- 哪怕只看到一段话，也可以预测下一段是什么。
- **留意**使你持续学习、水平持续上升。

◆ 对照时再追问一句：“这个差异来自什么约束或假设？”“学习研究中的“自我解释”，也关注把解题步骤与背后的原理联系起来。[Chi et al. \(1989\)](#)



**版本管理**

```
# 快速：GPT-6 发布了
- 幻觉更低了
- 品味提升了
- 价格变贵了
- 但设计上还是要写（我好像还可以再苟活几个月）
  - 当然这是因为面对了非常开放的 intent-spec-impl 空间
```

)

“理应当做到”的事就是能做到

```
- 直接杀死了比赛 🤖 具身赛道的人应该都在瑟瑟发抖
- 我对 OpenAI AGI Roadmap 的理解：一切对齐到数学逻辑推理
```

)

**Anyway... 我们还是要学习的**

对世界\*\*建立正确的概念和理解\*\*是具有发散和创造能力的前提。今天我们进入真正的软件工程任务，教大家如何从零开始构建一个项目。

首先，什么是一个\*\*软件工程项目\*\*？

```
- 一个目录，目录里有文件。这里混合了 intent, spec, impl, ...
- 经过 build，就能产出一个可交付的东西。无论是 npm run dev 的本地窗口，还是一个 exe
[[这里需要一张关于项目的图片，简洁一些]]
```

[63/lect/index.md](#) 18,16 109

## 两种“被细节击中”的体验，构成学习正反馈

- 讲者补上另外两种情况：
  1. 成熟、好的教科书方案比你想的方案更好、更精巧、更简洁。你会追问：“为什么他能想到？”——这个方案就变成你知识的一部分。
  2. 你的方法比它还更好。讲者说，这种体验可能一万次才出现一次；一旦出现，正反馈很强，你会突然想：“我能不能去做一个什么好玩的？”

- 讲者结论：在 **AI 时代**，更可以对细节保持注意力。例子：看 **Git**、看各种各样的 **Git repo**、读别人写的东西，甚至读 AI 生成的东西，都是感知细节的方式。

## 为什么要理解 Git：GitHub 让代码分发空前方便，并意外成为 AI 时代的底层基础设施

---

- 讲者开头问：为什么要 **有 Git**？
- 他先承认一个显然点：有 **GitHub** 以后，分发代码变得前所未有的方便。讲者认为，**GitHub** 创始人创立它时可能就想到了这一点。
- 但讲者认为，他们未必意识到：**GitHub** 竟然成为今天 **AI 时代** 的底层基础设施。
- 证据：**npm**，以及你各种 **npm** 的配置，大都在 **GitHub** 上。
- （整理者串联：讲者用 **npm** 生态说明，**Git/GitHub** 已不只是“放代码的地方”，而是许多工具链和 **AI 工作流** 的默认底座。）

## 推免面试观察：学生会说 Git 的用途，但 merge 冲突后的现象是盲点

---

- 讲者上周末参加 **推免面试**；他提醒听众大约一年或两年后也会到这种场合。
- 他问“什么是 **Git**”。大家多多少少能说两句：
  - 从 **Git** 上下载、分发代码；
  - 有同学能讲到 **Git** 管理项目目录的快照。
- 讲者态度：没必要因此把所有人都挂掉；能讲一些就可以，具体看能讲多少来评估。
- 问 **git rebase** 是什么，也在讲者意料之内（学生不熟或答不出符合预期）。
- 但讲者比较意外的是：他问“**git merge** 发生冲突以后会观察到什么”，大家似乎没有观察过，或答不上来。

## merge 冲突不是立刻被 resolve，而是把责任交到开发者手上

---

- 场景：作业中老师发布更新，你本地已经把东西改得面目全非；你让 **agent** 来，它自动啪啪修好，最后说“好了”，甚至测试通过；讲者自己也遇到 **merge** 出错，让 **agent** 帮修，看一眼结果看起来 **OK**。

- 但讲者认为，这时候**真正的注意力**才有用。若你愿意在冲突时停下来——尤其学生常想“我再学一点吧，反正一秒钟，学一下没坏处”——你会看到很神奇的东西：**很多等号、很多箭头**。
- 讲者给出的责任链：
  1. **merge 冲突发生**；
  2. 冲突不会立即被解决，不是自动 **resolve**；
  3. 解决责任落到**开发者**手上，曾经是每一个开发者；
  4. 开发者发现冲突后，就有义务解决冲突；
  5. 要解决冲突，他就必须**能看到冲突**。
- （整理者串联：因此，若让 agent 直接把冲突抹平，你就跳过了“看到冲突”和承担解决责任的过程，也跳过了讲者所说的细节学习机会。）

## 讲者的主张：AI 代修越顺手，越要在冲突处保留“看一眼”的习惯

---

- 讲者不是否认 agent 能修好；他描述的是自己也会让 agent 修，测试过了、看起来 OK。
- 但他强调：冲突时的等号、箭头不是噪音，而是 Git 把**解决责任和冲突现场**暴露给开发者的方式。
- 对学生而言，讲者认为在冲突处停一下的成本很低（“一秒钟”），却可能把一次自动修复变成一次对 Git 细节的感知训练。

小结：讲者把 Git merge 冲突从“让 agent 修掉的麻烦”改写成“把责任交回开发者、训练细节注意力的入口”。

---

## 第 10 部分

仙1-107老师



### Git: 听说过, 还是用过?

上周的保研面试

- 解释什么是 Git: 多多少少都能说两句。
- git rebase 是什么: 0/10。
- git merge 冲突以后, 会观察到什么?
  - 竟然没有人说出: 文本内容冲突时, 双方内容和冲突标记会直接写入工作区文件: git status 显示 both modified。

第一没用过, 第二没有留意

如果平时刷各种材料, 多多少少会刷到: 真正用过, 也要留意眼前发生的事。

♦ 观察入口: [打开冲突文件](#), [看看 Git 留下了什么](#)。

## 开发者必须解决冲突, 因此 Git 必须让冲突“可见”

- 讲者先给出义务: 开发者有义务把冲突解决掉。
- 逻辑链:
  1. 有冲突 → 开发者要解决。
  2. 要解决 → 他必须能看到冲突双方。
  3. 因此 Git 不能只说“对不起, 冲突了, 我不能合并”然后停在那里; 这会把决定负担全丢给使用者。
- 例子: 一个文件原来是 1111; 一个人把第二行改成 2, 另一个人把第二行改成 3。那这一行到底是 2 还是 3? Git 不能替人决定, 但也不能只报冲突后不管。
- 讲者认为, 如果 Git 只是“冲突了不能合并”然后躺在那, 对人的负担非常大: 人还得自己判断这一行应该放 2 还是放 3。

## Git 的冲突呈现遵循“把信息放到现场”的第一性原理

- 讲者说所有 first principle 都是连起来的; 自己之前无意中举的地铁标识例子, Git 也在用同一个原则。
- 原则是: 如果现在发生了一件事, 最好就在这个地方, 把它所有需要的信息全部放进来。
- 所以以文本方式看 Git 冲突时, 会看到: 等于号、多个尖括号、incoming / HEAD 之类的标记; 至少冲突的两部分代码同时显示在这里。

- 如果使用 IDE，上面还会多出按钮，比如 **accept incoming change**；可以在 IDE 里自己 **resolve**，决定谁进谁出；完成之后再创建一个 **commit**，把这个冲突 **resolve** 掉。
- 讲者认为这只需要一点注意力；只要有一两次经验，就会形成印象：至少能说出两边冲突的代码会显示在一起。

## 大家答得不好：原因一是没用过，二是没留意

---

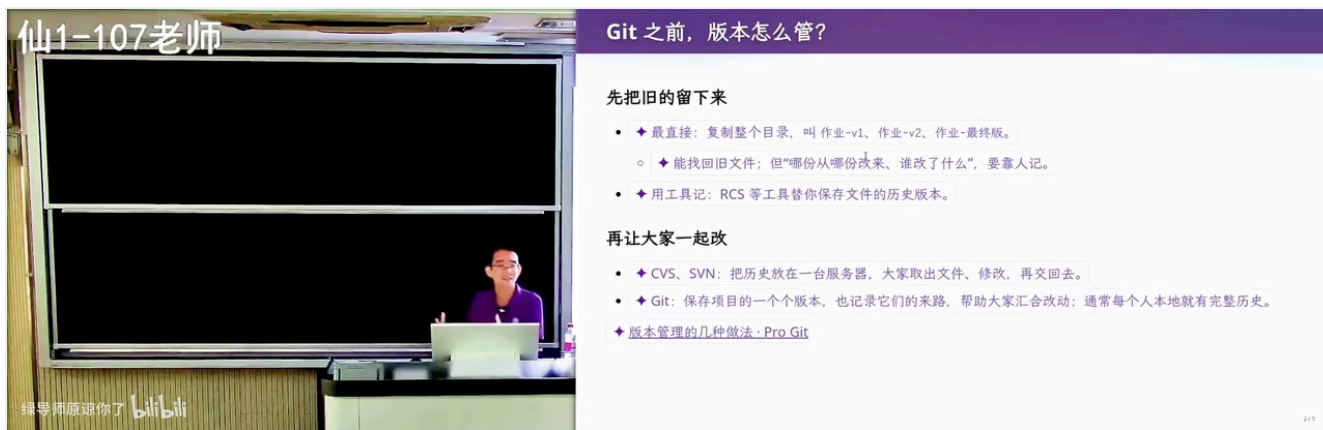
- 讲者说大家答得不好让他吃惊。他判断原因：
  1. 第一，没有用过、没有开发过。
  2. 即使用 A 证（音；疑指 AI/agent）开发，如果你让它在不同分支开发，也很容易遇到 **conflict**。
  3. 第二，还是没有留意。
- 所以比较重要的两件事：第一，多去用；第二，试图拥有更好的注意力。

## Git 的起源：从“复制目录保存旧版本”的需求到目录级版本管理

---

- 讲者先否定：在 U 盘上放一个目录、每次打个包的方式不行。
  - 但他的观察是：上学时同学们确实这样干——复制目录成 **作业 V1**、**作业 V2**（原音近 VR）、**作业最终版**。每次复制一个目录，非常直觉。
  - 这种方式可以找回旧文件。因为万一哪天手滑，或者不小心改错，尤其在新手阶段，误操作非常正常；人会想回到上一个版本，看看当时到底怎么写的。这不就是一个需求吗？
  - 从这样的需求出发，讲者认为你自己就能发明一个像 Git 这样的东西，因为本质上所谓的**就是版本管理**。
  - 讲者对项目的定义：项目就是一堆东西——代码、文档、**Intent spec**、**implementation**、各种东西、测试用例——全部搅在一起，放在一个目录里。
  - 接着的设想是：如果我们能够把**这个目录的版本管理起来**，那么就不再叫作业一、作业二、作业三，而是**只有一个作业**。
    - （整理者串联：也就是让版本历史由系统保存，而不是靠复制目录和文件名来区分。）
-

## 第 11 部分



## 目录版本化 + list/切换/回退 = 简化版 Git

- 讲者从一个思想实验开始：只有一个作业目录，不再叫“作业一、作业二、作业三”。
- 如果提供一个命令行工具，可以：
  - `list` 作业有哪些版本；
  - 把当前版本变成另一个版本；
  - 回到过去的某个版本。
- 讲者明确推出：这样你就已经发明了一个简化版 Git，虽然还没用到分支、合并、冲突这些概念。
- 讲者认为，分支、合并、冲突会随着使用自然被发明出来。

## 同步、同文件修改会逼出分支/冲突

- 用着用着，你可能会要和室友同步。
- 同步中出现“两个人一起改同一个文件”的情况。
- 就会坐下来想：一起改了同一个文件该怎么办？
- 于是就可以发明 Git（转录中作“get it”）相关机制。
- 讲者由此感叹/认为：这体现了人类的局限；在 Git 之前，还用过 SVN，你们可能听都没听过。

## SVN 存变化 vs Git 追求快速回到任意版本

- SVN 有 **check in / check out**。
- SVN 的设计：存储的是项目的变化。
  - 有最初版本；
  - 文件改一行，就存一个 **patch**；
  - 再改一行，再存一个 patch（转录作 PCH）。
- 讲者评价：这看起来没问题，甚至是非常高效的实现。
- 但讲者说，这种高效会随着项目增长把大家拖入泥潭。
- Git 的模型要干净得多。讲者的 **first principle** 是：我想先回到过去某一个版本，就应该马上、一秒钟回去。
- 反例逻辑：如果每次存的都是 **delta**（德尔塔），要回到早期版本，就得一个一个往回退。
- 可以用缓存等方法优化，但讲者认为这条路线确实像是走错了。
- 讲者链条：SVN 存 delta → 回早期版本需逐个回退 → 不符合“快速回到任意版本”的目标 → Git 的 **first principle** 更干净。

## Git 的历史：Linux 维护痛点催生 2005 年的 Git

---

- Git 的作者是 Linux 的作者。
- 1991~2002 年维护 Linux 时，用的是类似“作业 V1、作业 V2”的手工方式，在邮件列表上发补丁。
- 具体流程：
  1. 我改了 `a.c`，手里有一个 `a.c` 新版；
  2. 我把新版发到邮件列表；
  3. 维护者每天看邮件列表，把它放进核心仓库；
  4. 再去 **release** 这个版本。
- 之后还用其他工具 **BitKeeper**（原转录 [get it key / keeper]；此修正为整理者按上下文判断）。
- 直到 Linux 受不了，认为应该要一个**绝对简单、非常好用、概念正确**的版本控制工具。
- 结果：2005 年做了 **Git**。讲者强调它比我们想象中年轻。
- 有兴趣的同学可以看相关文档（原词 [PROPATE?])；讲者说不用再读，他读书时都看过这些文档。

## 项目级时间回溯是刚需

---

- 讲者最后指出：我们需要项目级的时间回溯。
- 因为我们都会犯错误，所以只要能进入 Git 的历史里回溯/恢复。

小结：讲者强调 Git 的根本出发点是项目级时间回溯；分支、合并、冲突等概念是后续使用中自然长出来的。

## 第 12 部分



仙1-107老师

我们要的是项目级“回溯时间”

人和 Agent 都会犯错误

- 设计会被改掉，对的代码会被改掉.....
- 哪怕手滑把文件删掉了，只要已保存在 Git 历史中，就能捞回来。

Git 管理的是项目的快照

- 一次提交，记下纳入版本管理的文件当时的内容，以及它们在目录里的位置。
- 为什么要以项目为单位？代码和配置一起改过，只恢复旧代码、留下新配置，可能拼出一个从未正常运行过的版本。
- “存过盘”不等于“存进历史”：希望将来回到状态，需要提交保存。

快照与提交 · Pro Git



仙1-107老师

学习知识变容易以后，\*\*学习意识\*\*的重要性提高了：公众号上满天都是 github 链接。你会看到\*\*大量别人的项目\*\*——这些就是你的学习素材。哪怕没有人告诉你哪个好哪个不好，看得多了也有感觉了。（这就是“上课耽误学习”的本质：课外的信息量、信息密度巨大，但我们从未教过“如何用好”）

- 学习建议：大家需要对细节的“注意力”。每一次阅读别人的东西（甚至是 AI 生成的东西），都有“感知细节”——读论文的时候有一个 virtually implement 技巧。在看到问题以后，先问自己是怎么想的，如何 approach 问题，如果和作者一致一很好；如果不一致，就是一个学习的机会。这个原则适用于你看到的任何东西，哪怕看到一段话，也可以预测下一段是什么：\*\*留意\*\*使你持续学习、水平持续上升。

## Git

上周参加了保研面试。问了以下问题：

- 解释什么是 git [这个多多少少都能说两句]
- git rebase 是什么 (0/10)
- git merge conflict 以后会观察到什么？
  - 竟然有人说出 both modified 会直接写入工作区

说明：第一没用过，第二没有留意（如果平时刷各种材料，多多少少会刷到的）

[[给一个Git的简介和历史，尤其是Git之前怎么管理版本，给历史配1个图。要用最不烧脑子、最直接、最朴素的概念来清楚地解释，用你的方式讲，不要看我的]]

你最需要建立的概念是：Git 管理的是项目的\*\*快照\*\*，但不应该是通过背诵记忆的：人和 Agent 都会犯错误，设计会被改掉，对的代码被改掉.....我们希望项目级“回溯时间”的能力——这就是我们要的。哪怕手滑把文件删掉了，只要历史上存在，就能捞回来。

- 符合我上一次课讲学习的模型：“现在的你”是一个知识网络，学习的方式是自然地“外推”它，这样可以帮你记忆。

p3/lect/index.md 48,58-55 25%

## 项目级时间回溯：会犯错，所以要用 Git 历史把东西捞回来

- 讲者先立目标：需要 项目级的时间回溯。
- 理由：我们都会犯错误。
- 因此，只要能去 Git 的历史里把误删、误改的东西“捞回来”，就可以了。

逻辑链：

1. 项目会犯错。

- 2. 需要项目级时间回溯。
- 3. 只要能进 Git 历史把东西捞回来，就足够。→（整理者串联）所以 Git 的历史不是装饰，而是恢复机制。

## 从已有经验推出下一个概念：讲者把这称为数学推理

---

- 讲者建议用这种方式推进：从已有经验出发，明确“你知道什么”“你想要什么”，然后推出“下一个概念应该是什么样的”。
- 讲者说：这就是 **数学的推理**。
- 他补充自己上课时，每一句话都和之前有逻辑关联；这些逻辑关联多多少少都有数学联系。
- 由此，OpenAI（原转录：open a）认为要先把数学训好；训好以后，它所有问题可能都能解决。讲者认为这件事可能真的要成真；讲者自己觉得“品味”（taste）可能也能解决。

## git init：把一个普通项目变成 Git 版本管理的仓库

---

- 讲者现场建一个空的 **terminal**，讲什么是 Git（原转录多处像 get it / kit）。
- 开启项目：`git init`（原转录：get in it / get it event）。
- `git init` 会输出一些奇怪信息；讲者说这些信息不重要。
- 面对不懂的信息有两种选择：
  1. 让 **agent** 帮你做、帮你解决、回答你不懂的问题；
  2. 把它忽略掉。
- 讲者认为前者显然是更高效的学习方法，也更好。
- 结果：得到一个空仓库，或者说得到一个 **.git 目录**。`.git` 默认不显示，所以看不到；讲者说这就是全部了。
- 做完 `git init` 后，项目就变成被 Git 版本管理的 **仓库**。
- 后续能力：可以从远端下载代码，也可以把代码推到远端。

## 基本命令链：`git add` → `git status`，Git 的核心是项目快照

---

- 例子：有一个 `hello.c`。讲者说随便，你们喜欢写 C 好。
- 运行 `git add hello.c`。

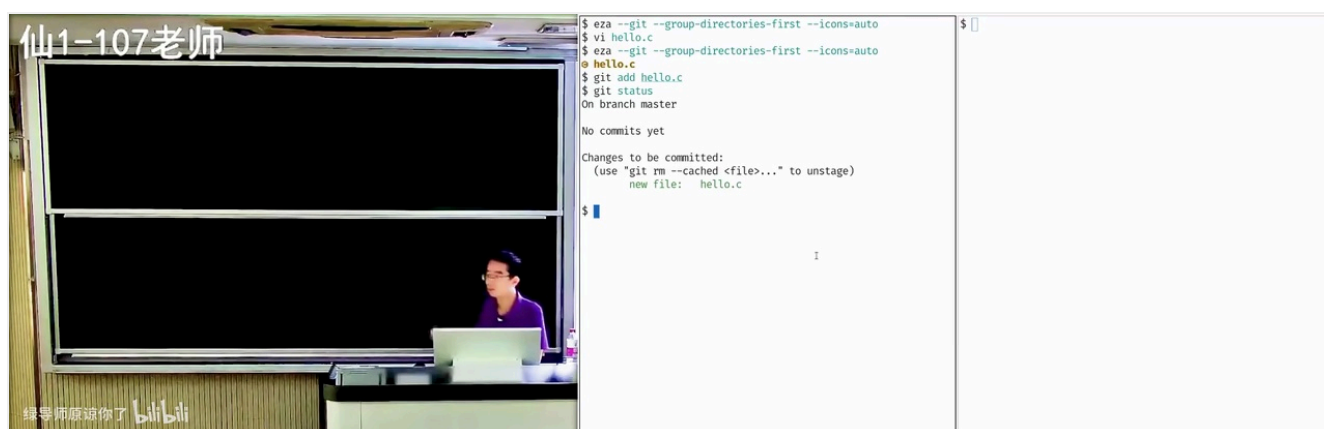
- 再用 `git status` 可以看到当前 工作区 里有 `hello.c`，并有一些提示。
- 讲者强调 Git 很重要的一点：创建 项目快照，使你能在任意时间之间穿梭。
- 可以创建一个 有消息的快照；消息很重要，体现 软件工程的智慧。
- （整理者串联）所以 `commit message` 不是形式，而是让快照可理解、可回溯。

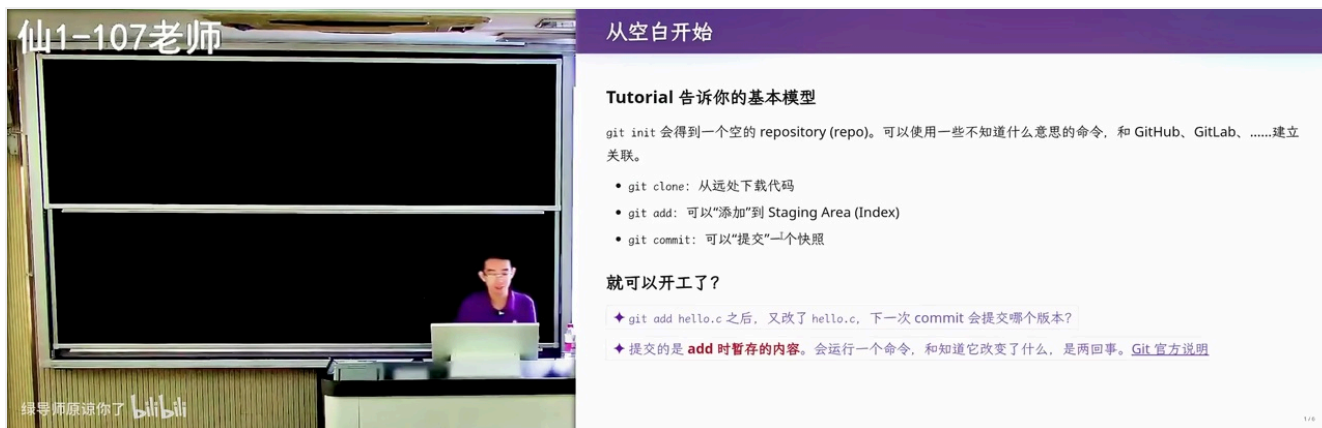
## 不要用“作业 V1/V2/V3”代替提交：每次改动都应立即 `git commit`

- 以前用复制粘贴的方式命名：目录一、项目版本 V1/V2/V3、作业 V1/V2/V3。
- Git 的世界不是这样。讲者看到很多同学在用 Git 时，把 Git 当成作业 V1/V2/V3 来用；这不是 Git 推荐的 **practice**。
- 推荐 practice 来自长期软件工程实践经验：
  - 对项目做任何改动后——改文件、修 bug、增加功能、调整脚本、调整文档——都应该立即做一次 `git commit`。
- 演示 `git commit` 时打开 Nano（讲者：哎呀这怎么是 Nano）；可以在文本编辑器里写“我这是做了什么”。

逻辑链：改动（改文件 / 修 bug / 加功能 / 调脚本 / 调文档）→ 立即 `git commit` → 得到带消息的快照 → 可以在任意时间之间穿梭。

## 第 13 部分





## 改动一发生就 `git commit`，message 要写清你做了什么

- 讲者主张：只要你调整了某个文档，就应该立即做一次 `git commit`。
- 演示：他打开文本编辑器（惊讶“哎呀这怎么是 Nano 啊”），写一段文字说明“我做了什么”，保存后提交成功。
- 他说这些都可以在现有项目里看到：别人是怎么协作的。有些提交较大，放了一个 feature；有些提交较小，只 fix 很少的东西。
- 讲者由此认为：面对一个新工具，理解它的 **best practice** 需要花时间。

## 不能把一切交给 AI：要知道为什么做、何时 `git add / git commit`、message 怎么写

- 讲者认为，不能把所有事情都交给 AI；你多多少少还要知道我们为什么要做这件事，什么时候应该 `git add`，什么时候应该 `git commit`，commit message 应该怎么写。
- 学习方式：碰到新东西时马上追问：“你刚才做的是不是符合 **best practice**？”比如刚做完一个 `git commit`，就问“这个符合 best practice 吗？”
- 他举例：向豆包输入法提问只需一秒钟，但它会给很多解释。它说这个 commit 不太符合 best practice：主要问题在提交信息，其次代码本身也有问题；提交信息的可读性有问题。
- 讲者故意写了一个 `I added some cool stuff`，完全看不出做了什么。

## Git 存快照，但项目历史被理解为 V1 → V2 的修改，所以 message 必须描述改动

---

- 讲者说：Git 本质上存放的是快照，但我们的项目本质上还是从一个版本 V1，经过一次修改变到 V2。
- 因此 Git 鼓励你在 message 里描述你改了些什么。
- 讲者的链：message 写清改动 → 一直读提交记录 → 就能知道项目怎么发展到现在：修了多少个 bug、加了什么需求、经历了什么样的重构；可以在 message 里看到项目历史的轨迹。
- 讲者认为这也是好的软件工程 practice，使别人更容易理解你的项目。

## 极端反例：10,000 条 “I did some cool stuff” 会让 commit message 变成 slop

---

- 如果每一个提交都是 I add some cool stuff，想想 10,000 个提交以后会怎样：commit message 就变成 slop 了。
- 如果 10,000 条都是 I did some cool stuff，那 commit message 这个特性就不应该存在；不要叫 commit message，直接从 V1 到 V2 就行了。
- 讲者说，你会在不同时候使用不同的 commit message。比如一个版本要收尾了，可能会写一个 commit message 说“我要 finalize 这个版本”。

## 空提交的边界：普通 Git 拒绝，agent 可以做，但理论可行 ≠ best practice

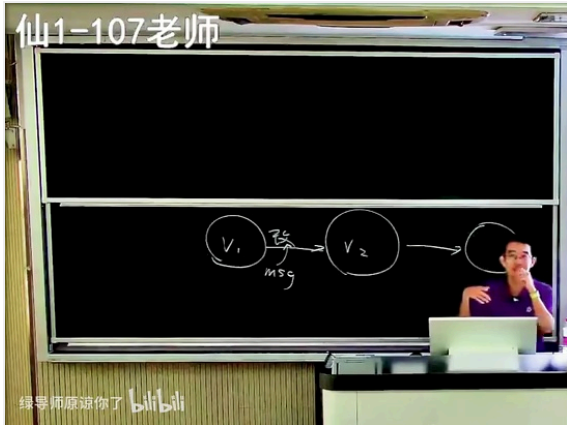
---

- 讲者提出一个边界情形：不改任何代码，V1 和 V2 完全相同，再做一个提交。
- 普通 Git 会拒绝这个空的、没有修改的提交；但 agent 可以。讲者说，他让 agent 做一个提交，并要求它在不修改的情况下提交，agent 能做到——讲者说“它知道，它有知识”。
- 讲者强调需要保持注意力，尝试追问：“从理论上说应该可以吧？”比如“我就要发布一个版本了”这种提交。
- 但讲者认为：这个不是 best practice。

小结：commit 不是随手保存；message 要让项目历史可读，AI 可以快速答疑，但 best practice 仍要自己追问和判断。

## 第 14 部分

仙1-107老师



绿导师原谅你了 bilibili

```
On branch master
nothing to commit, working tree clean
bccfe44 I did some cool stuffs

Took 0.0s

Thinking...

$ cd /tmp/hello 66 git commit --allow-empty -m "Empty
commit" 66 git log --oneline -3

[master a227ea4] Empty commit
a227ea4 Empty commit
bccfe44 I did some cool stuffs

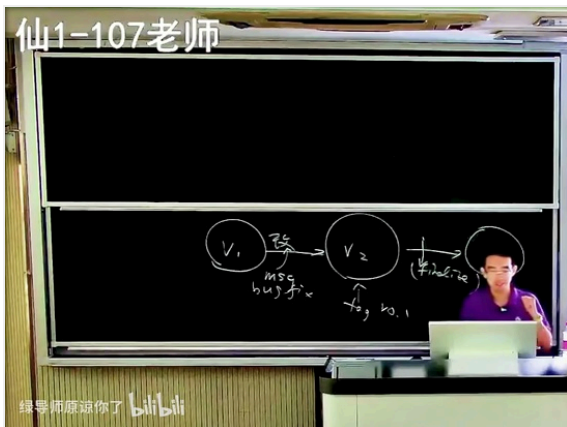
Took 0.0s

已创建空提交 a227ea4 (git commit --allow-empty) . 仓库内容零
改动。仅新增一条提交记录。

Recap: 用 git commit --allow-empty 可在工作区无改动时仍产
生一次提交，常用于触发 CI 或留档。

/tmp/hello (master)
#333 4305 R7.3k CH94.3% $0.000 0.3%/1.0M (auto) deepseek-v4-f
```

仙1-107老师



绿导师原谅你了 bilibili

### 获取下一级知识

学会追问，学习才进入正轨

绝大部分人都倒在了“获取下一级知识”的路上。

- ◆ “能运行”以后：为什么别人还要写 .gitignore?
- ◆ “文件都在”以后：为什么换一台电脑就编译不了？

### Best Practice 从哪里来？

软件和系统都有**约定俗成**的 best practice，而每一个都是血的教训，或者对旧设计忍无可忍的产物。

- ◆ 把源码和编译产物一起提交：源码改了，产物忘了更新——同一件事有了两份互相矛盾的状态。
- ◆ 把自己的绝对路径写进配置：在我电脑上能跑，到了同学那里就失效。
- ◆ “目录里有什么”和“项目需要共同保存什么”，是两个问题。

## 空提交不是 best practice，但在“无改动却要触发流程”时偶一为之

讲者先提出一个“理论上应该可以”的做法：故意做一个 空提交 (empty commit)。但他马上限定：这不是 best practice，所以尽量不要做。

空提交有用的场景：

1. 当前没有代码改动。
2. 故意产生一次提交事件。
3. 这次事件可触发 CI 或流水线 (ASR: 流达)。（整理者串联：CI/流水线通常监听 commit/push 事件；讲者只说到“产生提交来触发”，未展开触发机制。）

发布/收尾场景的推理链：

- A：上一次已经修完最后一个 bug，得到一个可以 release 的版本。
- B：Git 可以在这里打 **tag**，比如 **V0.1**，说“这是 V0.1”。
- C：讲者说也可以强行做一个无改动的提交，说“我要 **finalize**”，为了 **0.1 freeze** 所有东西。
- D：这样就把 **bug fix 的提交**和 **finalize 的提交**分开了。
- 讲者结论：这件事偶尔可以允许；但不能创建大量这样的空提交。

## Git 命令只是表面，真正难的是水面下的 best practices

---

讲者认为：学了 `git push`、`git pull` 等基础命令（原文 ASR 含糊）后，好像就会用 Git；但真正在水面下的冰山，是各种 **best practice**。每一个 best practice 都是“血和泪的教训”。

他举出一个要追问的问题：**为什么 commit message 要那样写？** 他建议反过来问 AI，或跟更智能的模型讨论（ASR: 更有质检模型），问这样做有什么好处。

讲者的个人观点：

- 很多时候模型表现比他好。
- 知识可能是内化在他脑子里的；给他两小时，他也可能写出很好的“为什么要做这件事”。
- 但他的智力、精力、注意力有限。上课时要专注和大家上课，大脑中做思考的容量不够，所以讲一个概念时讲的不是最好的版本。
- 因此，回头可以把“为什么 commit message 要这样写”这类问题拿去和 ChatGPT 聊。讲者把这比作求助一个自己无法掌控、但“好像知道世界上所有东西”的东西；它能把这些概念讲清楚。
- 这些约定俗成的 best practice 是核心。

## `git add .` 合法，但不自动等于 best practice

---

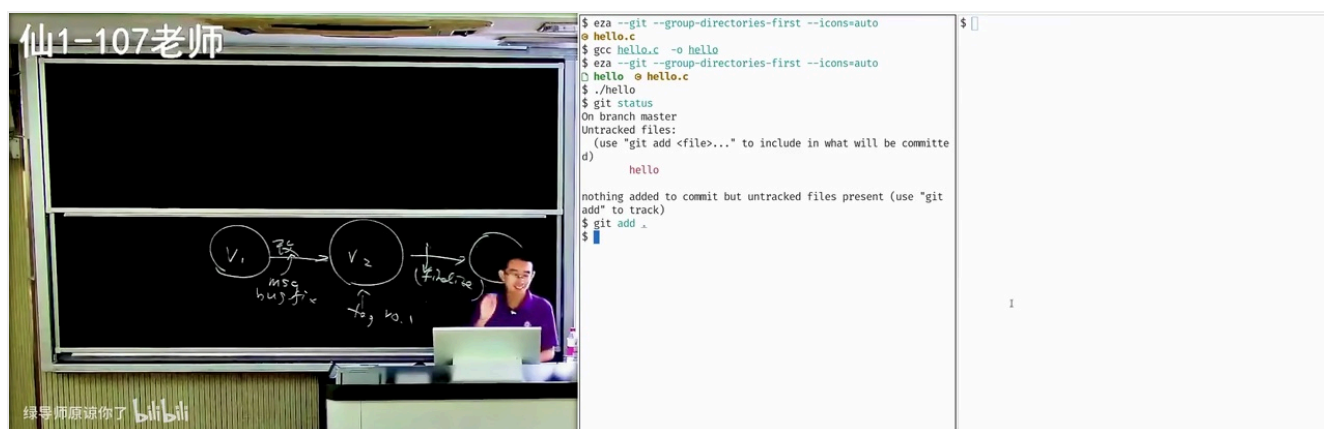
讲者举一个“不是 best practice”的常见做法：

- 场景：有一个 **untracked file**，tutorial 里会看到一个很好用的命令 `git add .`。

- 如果你改了三五个文件，比如 `a.c`、`b.c`、`c.c`，你不需要写 `git add a.c b.c c.c`，直接用 `git add .`。
- 然后就可以提交。
- Git 不阻止你这么做；这是完全合法的。
- 讲者接着问：那么 Git 提交里有什么不符合 best practice 的吗？——这个问题引向下一部分。

小结：空提交只在无改动却要触发/标记 release 的偶发场景可接受；Git 的难点不在命令，而在 best practice。

## 第 15 部分



## 合法 ≠ 符合 best practice：让工具审查 Git 提交

- Git 本身不阻止你把编译产物等东西提交上去，这是完全合法的。
- 讲者接着让工具检查：这些 Git 提交有没有不符合 best practice 的地方。
- 工具会检查，并按严重程度排序反馈。讲者认为这种排序“还是很聪明的”。

## 最高严重度：提交信息毫无信息量

- 工具指出：提交信息毫无信息量，甚至是主观废话。讲者半开玩笑地说，已经等着它“开骂”了。
- 工具随后给出应该怎么做：
  - 用祈使句；

- 用现在时；
- 描述变更动机。
- 例子： `fix off-by-one ...` 这类写法。
- 对比坏写法：像“你的 C 里面有这种不好的 practice，我现在 fix 了”这种叙述，不如直接说明修了什么。
- 工具还提到常用的约定：**Conventional Commits** 前缀值得考虑。讲者补充：这种 commit 方式在更多项目里也能看到。
- 讲者确认：提交信息毫无信息量是首要问题、最重要的问题。

## 第二严重：提交粒度与内容混杂，尤其是编译产物入库

---

- 工具第二个批评的是提交粒度和内容混杂。讲者说“他开始批评我了”。
- 核心问题之一： `hello` 这类编译产物根本不应该进版本库。
- 正确做法：
  - 加进 `.gitignore`；
  - 由构建系统产出。
- 讲者指出：这和你们作业 V1、V2 的使用方式不一样。
  - 作业 V1 里 `.exe` 一定在里面；
  - 作业 V2 里 `.exe` 也在里面；
  - 但 Git 为了防止版本库膨胀，不应这样用。

## 编译产物入库的后果链

---

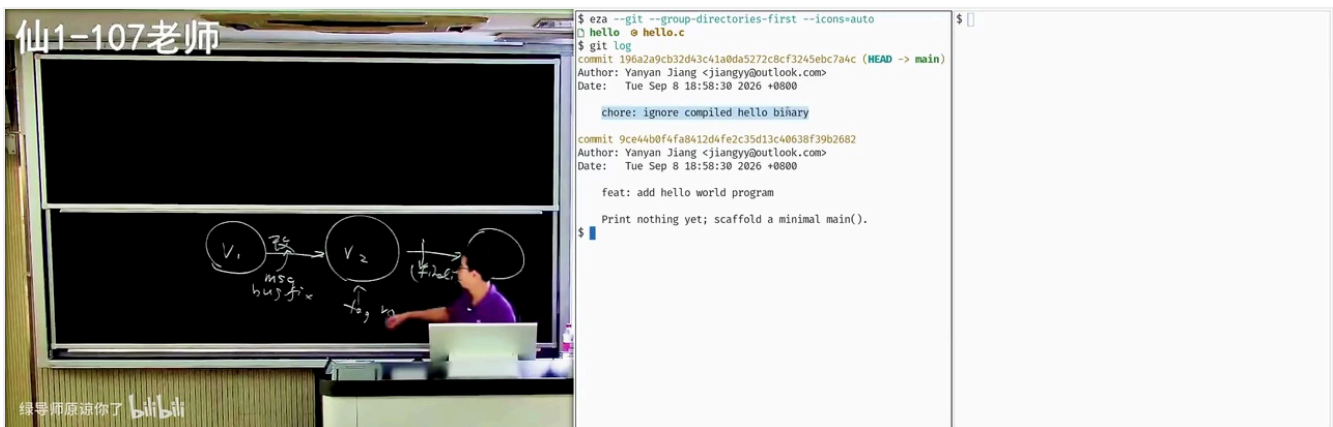
1. 项目可能有 1 万个文件、100 个合作者。
2. 每个合作者的环境都不一样。
3. 每个环境 build 出来的东西也不一样。
4. 编译产生的文件每次都可能不同：可能带日期，甚至带 hash，带本机的指纹。
5. 所以同一个二进制文件，你这里一个 `hello`，对方那里一个 `hello`，哪怕源代码相同，编译出来也可能不同；如果源代码不同，就更不用说了。
6. 把它放进版本库，下一次就会产生冲突。
7. **二进制冲突很难合并**。讲者还举例：如果生成的是图片，也会遇到“怎么合并”的特殊问题。

→ 因此才有这一堆 Git best practice：生成物不要入库，用 `.gitignore`，让构建系统重新产出。

## 工具不只批评，还直接重写历史

- 讲者让工具继续行动，并说结果可能会出乎意料，自己也不知道会发生什么。
- 等待过程中讲者一度以为网断了，但实际没有；工具花了一些比较长的时间思考。
- 工具说：
  - 丢弃空提交和编译产物两类历史；
  - 新建 `main` 分支，重写为两个有意义、单一职责的提交。
- 工具还贴心地发现历史没有 `remote`：
  - 也就是说仓库没有和另外一边同步，不会有别人持有；
  - 讲者解释：如果别人也持有，重写历史就是比较糟糕的事情；
  - 因为没有别人，所以工具把整个项目改了。
- 改写后可以观察到，它换了一套 `commit` 方式：
  - 有一个 **headline message**；
  - headline message 后面还有一些内容。

## 第 16 部分



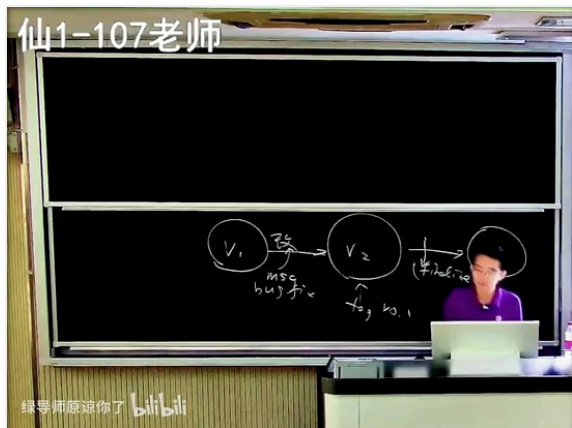
```
$ eza --git --group-directories-first --icons=auto
hello @ hello.c
$ git log
commit 196a2a9cb32d43c41a8da5772c8cf3245ebc7a4c (HEAD -> main)
Author: Yanyan Jiang <jiangyy@outlook.com>
Date: Tue Sep 8 18:58:30 2026 +0800

    chore: ignore compiled hello binary

commit 9ce44b0f4fa8412d4fe2c35d13c40638f39b2682
Author: Yanyan Jiang <jiangyy@outlook.com>
Date: Tue Sep 8 18:58:30 2026 +0800

    feat: add hello world program

    Print nothing yet; scaffold a minimal main().
$
```



## 获取下一级知识

### 学会追问，学习才进入正轨

绝大部分人都倒在了“获取下一级知识”的路上。

- ◆ “能运行”以后：为什么别人还要写 .gitignore?
- ◆ “文件都在”以后：为什么换一台电脑就编译不了?

### Best Practice 从哪里来?

软件和系统都有**约定俗成**的 best practice，而每一个都是血的教训，或者对旧设计忍无可忍的产物。

- ◆ 把源码和编译产物一起提交：源码改了，产物忘了更新——同一件事有了两份互相矛盾的状态。
- ◆ 把自己的绝对路径写进配置：在我电脑上能跑，到了同学那里就失效。
- ◆ “目录里有什么”和“项目需要共同保存什么”，是两个问题。

2/6

## Commit message 的 headline 不只是标题：fix #3 能触发 GitHub 自动关闭 issue

- 讲者说，headline message 后面还有好多好玩的东西。
- 例：在提交信息里写 fix #3（原文“fix 井三”），GitHub 会自动 close 对应 issue。
- 讲者判断：这些功能只有反复使用，才能看出初学者和有经验者的区别。

## Agent 时代：知道 Git 能干什么，比记住具体命令更重要

- 讲者认为现在是 agent 时代，你们是 agent native，不会用古法；git status 这类操作在讲者看来已经是“古代手艺”。
- 逻辑链：
  1. 你不需要知道具体命令是什么；
  2. 但你要有概念：Git 能干什么；
  3. 然后让 agent 用符合 best practice 的方式去做。
- 风险：这样会分不清你是真的自己能干，还是脱节、其实干不了。讲者说，也许这也不重要。
- 为了提高学习效率，讲者建议：做任何 key 操作之前，逐个检查文件变化。

## 演示：让模型提交不能编译的代码，模型会拒绝并解释规范

- 讲者随便试一个不能编译的代码，然后让模型做提交；他自己也不知道模型会给什么。
- 模型表现得很“聪明”：它不建议直接提交。
- 讲者推测原因：模型有工程经验，见过很多好的工程项目，也读过很多软件工程的书。
- 模型给出的理由：
  - Git 提交应当是完整自洽的快照；
  - 编译不过的代码相当于让仓库处于坏状态。
- 模型还顺手修好了代码，并给出解释；其中有些解释讲者知道，但即兴上课时想不到。

- 对照人的倾向：改作业时容易想“改一个字不要紧，直接提交吧”；讲者说模型比你聪明。
- 这个演示里可能用了 **system prompt**，例如要求说中文，或说明“我在上课，所以要讲得更详细一点”。

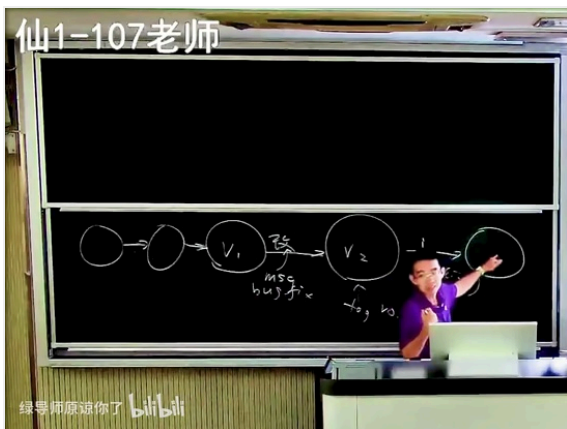
## git blame：每行代码都能追溯到某次提交

- 讲者很喜欢 `git blame`。
- 例：对 `hello.c`，它会告诉你每一行代码是在哪一次提交、哪个地方改的。
- 讲者说：即使只有线性历史，也能干很多有趣的事情。
- 他的感慨：AI agent 的一句话就把 Git 给打开了。

## git bisect / 二分查找：把提交历史当数组来定位 bug

- 讲者提到 `git bisect`（原文 ASR: get it by sect）；Git 还有 `git blame` 这个很有意思的功能。
- 二分查找的背景：大家可能在保研或工作面试里被要求手写二分查找；它面对的是一个很长的数组。
- 类比：可以把 **Git 提交历史**想象成一个很长的数组。
- 场景：当前版本发现一个 bug，但这个 bug 可能不是当前代码引起的，而是过去某个地方引起的。
- 这时需要一个测试用例，或者任何能检查出这个 bug 的东西。

## 第 17 部分



```

error: 'a' undeclared → a 没被声明
error: expected ';' → 语句缺分号
...

为什么不符合 best practice?
- Git 提交应当是一个“完整、自治”的快照。提交编译不过的代码，
  会让仓库处于“坏状态”。
- 如果有人之后用 git bisect (二分定位 bug) 或 git checkout
  回到这次提交，拿到的是一个编译失败的版本，排查问题会很痛苦。
- 一条广泛接受的原则：每次提交的代码应当能通过构建/测试（至少
  主分支如此）。CI（持续集成）工具也会基于这条假设工作。

相关概念：a 这里的报错本质是—C 语言里每一条语句必须以 ; 结
  尾，而且 a 本身既未声明为变量，也不是表达式，属于非法语句。
  改动看起来是“打字到一半”，可能是该操作。

#### 文件 2: AGENTS.md (未跟踪) — 可以提交，但建议先想清楚
内容是一段对合作方（比如 AI 助手）的行为约定：“做 git 操作前
  逐个检查变化并解释”。

为什么符合 best practice?
- 把项目约定作为文档放进仓库，新加入协作的人都能看到，避免约
  定只存在于某个人脑子里—这本身就是好习惯。
- 很多仓库会把 AI 协作规范（如 .cursorsrules, AGENTS.md）版本
  化，方便团队共享。

需要你决定的两点：

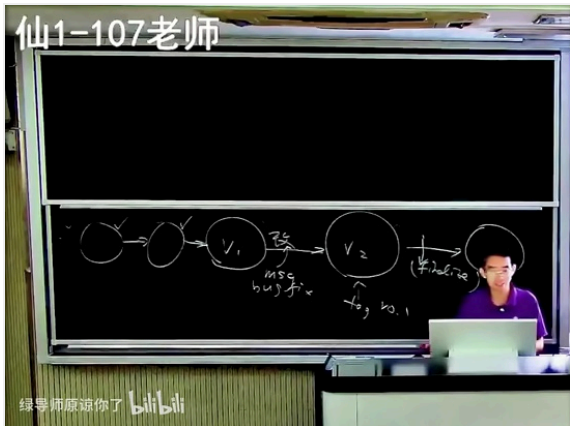
```

```

default: off)
-w          ignore whitespace differences
--[no-]ignore-rev <rev>      ignore <rev> when blaming
--[no-]ignore-revs-file <file>
                                ignore revisions from <file>
                                color redundant metadata from previo
us line differently
--[no-]color-lines          ignore revisions from <file>
--[no-]color-by-age         color lines by age
--[no-]minimal              spend extra cycles to find better ma
tch
-S <file>                  use revisions from <file> instead of
calling git-rev-list
--[no-]contents <file>    use <file>'s contents as the final i
mage
-C[<score>]                find line copies within and across f
iles
-M[<score>]                find line movements within and across
files
-L <range>                 process only line range <start>,<end
> or function :<funcname>
--[no-]abbrev[=<n>]        use <n> digits to display object nam
es

$ git blame hello.c
^9ce44b0 (Yanyan Jiang      2026-09-08 18:58:30 +0800 1) int m
ain() {
000000000 (Not Committed Yet 2026-09-08 19:12:16 +0800 2)  a
^9ce44b0 (Yanyan Jiang      2026-09-08 18:58:30 +0800 3) }

```



多问一句：为什么、该不该？

哪怕只是放一个 **hello.c**

缺少足够的知识，也会犯错误。这时候，对细节的“注意力”就非常重要了。

“ AGENTS.md: 在做任何 git 操作之前，都逐个文件检查变化，并为我 (新手) 解释是否符合 best practice，为什么，以及相关的概念。

让解释长出下一次判断的能力

- ◆ **hello.c**: 程序的源码，要保存。
- ◆ **hello**: 如果是编译出来的可执行文件，通常不提交：保存源码和构建方法，让别人也能生成它。
- ◆ 接着追问：这个文件是项目的输入，还是构建、运行留下的产物？别人拿到仓库，是否具备重建它的条件？

3/6

## 有能检查 bug 的测试后，git 历史可被当成“0...0 1...1”数组做二分

- 前提：bug 可能是过去某一个地方引起的；你有一个测试用例，或任何能检查出这个 bug 的东西。
- 讲者给出的推理链：
  1. git 历史里有一些提交 OK，有一些提交不 OK。
  2. 某一刻引入 bug 后，从某一时刻开始这些提交都不能通过。
  3. 在这个时刻之前的每一个提交应该都是 OK 的。
  4. 所以状态序列是：前缀全是 0，后缀全是 1。
  5. 要找的是第一个 1。
  6. 二分查找能做的事，就可以对整个 git 历史做二分。
- 讲者结论：有可检查 bug 的东西，就能用二分查找定位引入 bug 的提交。

## 定位到“错了”之后，git blame 追到谁改了这一行、发生了什么

- 讲者还提到 **git blame**（转录“get blame”）。
- 你知道错了以后，可以知道是谁改了这一行代码，以及发生了什么。
- （整理者串联）它回答的是某行代码的变更来源，与二分定位首个坏提交互补。

## 软件工程实践：讲者主张少在课上讲、多让实践和 agent 教，但构建与测试底线要反复强化

- 讲者认为软件工程可能不需要在课上讲那么多；时间要留给学生，在实践当中由 agent 来告诉你。

- 但有一类基础要求必须反复强化：**每次提交的代码应该能够通过构建和测试**。
- 讲者说，大家也会觉得“OK，应该是这样”，但反复强化、潜移默化以后才真正 OK。
- 讲者又提到 **AGENTS.md**（转录“A证的MD”，疑为 AGENTS.md）可以提交，符合 **best practice**；他不再展开。

## 新同学进组的“高血压时刻”：git 卫生问题清单

---

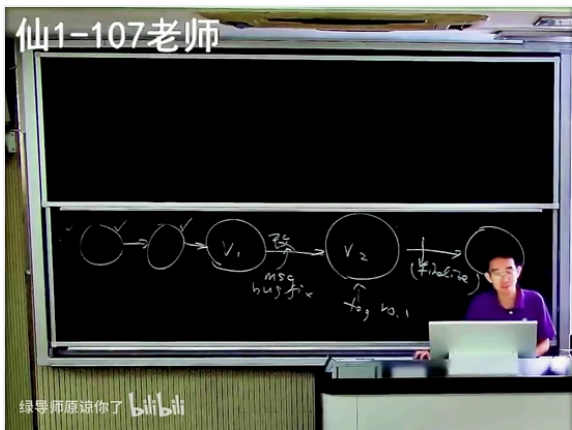
- 讲者说，在 LM/LLM（大模型）之前，一般有新同学进组都会出现一些“高血压时刻”，主题就是使用 git 的 best practice。
- 典型场景：欢迎加入研究项目 → 开一个 **git repo**（转录“get it ripple”）→ 第一周任务很简单，搭一个架子，把一两个东西跑起来。
- 讲者打开那个 git 项目后，进入高血压环节：
  - **dist 或 log** 被提交：log 也是生成文件，不应该提交。
  - **个人配置** 被提交：比如 VS Code 的个人设置，甚至里面有 C 盘下某个东西的硬编码路径；显然在任何一个其他地方都没有办法复现。
  - **active API key**：讲者在审稿时看到一些处于 active 状态的 API key，这是更高血压；当他发现 API key 是 **DeepSeek**（转录“deep pk”）时，就知道“应该是一个中国兄弟”（笑话）。
  - **.DS\_Store**：在苹果系统上用 **Finder**（转录“final”）打开一个目录，会自动留下一些索引，大概记录你把哪些子目录展开了，这些放在 **.DS\_Store** 文件里。
    - 它因为是点开头，所以默认隐藏；讲者说自己的 **ls** 还贴心地给它一个苹果图标，所以你看不到。
    - （整理者串联）若没有 **.gitignore** 排除它（原音频此处含糊），就可能被 git 跟踪并误提交。
    - 转录在“然后你看不到的时候”中断。

小结：讲者的链条是：可检查 bug → git 历史二分定位首个坏提交 → git blame 追到变更来源；而 git 卫生（生成文件、个人配置、密钥、.DS\_Store）是必须通过实践反复强化的底线。

---

## 第 18 部分

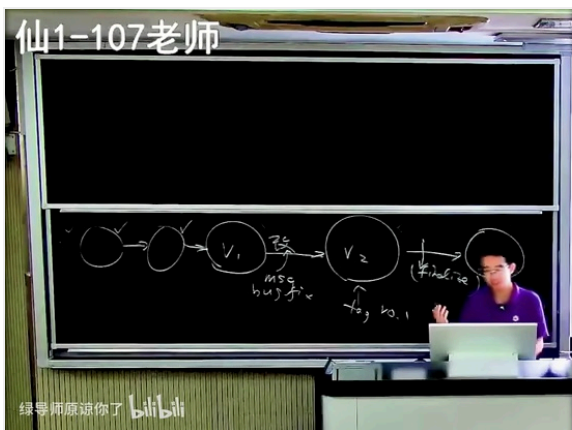
仙1-107老师



绿导师原谅你了 bilibili

```
$ eza --git --group-directories-first --icons=auto
= AGENTS.md  hello  hello.c
$ eza --git --group-directories-first --icons=auto -l
.rw-rw-r-- 153 jyy 8 Sep 19:10 -H = AGENTS.md
.rwxrwxr-x 16k jyy 8 Sep 18:54 -I  hello
.rw-rw-r-- 19 jyy 8 Sep 19:10 -H = hello.c
$ eza --git --group-directories-first --icons=auto
= AGENTS.md  hello  hello.c
$ eza --git --group-directories-first --icons=auto -al
drwxrwxr-x  - jyy 8 Sep 19:16 -I = .git
.rw-rw-r--  0 jyy 8 Sep 19:16 -H = .DS_Store
.rw-rw-r--  7 jyy 8 Sep 18:58 -- = .gitignore
.rw-rw-r-- 153 jyy 8 Sep 19:10 -H = AGENTS.md
.rwxrwxr-x 16k jyy 8 Sep 18:54 -I  hello
.rw-rw-r-- 19 jyy 8 Sep 19:10 -H = hello.c
$
```

仙1-107老师



绿导师原谅你了 bilibili

进组以后的高血压时刻

我和许多研究生新生都说过

- 生成的文件不提交 (dist/, log, .....
- 个人的配置不提交 (.vscode 中的个人设置等)
- ♦ 判断的是用途：团队共享、可移植的配置可以提交，不能只看目录名。
- 密钥不能提交 (审稿的时候见过好多)
- 临时文件不提交 (.DS\_Store, .....
- 规则是活的：package lock, generated header, ...

忽略不等于遗忘

- ♦ `.gitignore` 不会让已经被跟踪的文件自动退出版本管理。
- ♦ 密钥一旦泄露，删掉当前文件也不能收回别人拿到的副本：应先撤销或轮换凭据，再处理历史。[GitHub 指](#)
- 查

### `.DS_Store` 默认隐藏，但 `git add .` 会让它进入 Git 追踪

- 讲者指出：`.DS_Store` 默认隐藏，看不见；系统还很“贴心”地给它一个苹果图标。
- 逻辑链：
  1. `.DS_Store` 默认隐藏，看不见。
  2. 执行 `git add .` 时，带点文件仍可能被加进来。
  3. `.DS_Store` 会被 Git 追踪；而它是经常变化的临时文件，本不该被提交。
- 讲者判断：这个临时文件被纳入版本控制“有点糟糕”。

### Git 默认把带点文件纳入并非荒谬：点文件常是约定配置

- 讲者解释：带点文件可能是配置文件、`.github`、`workflow` 等约定俗成的东西。
- 所以默认把带点的文件也放进来，有合理性。

- （整理者串联）因此 `.DS_Store` 的问题不在“带点”，而在它作为频繁变化的临时文件不该被提交。

## “生成文件都不提交”不是绝对规则：`package-lock.json` 与生成 header 是反例

---

- 讲者强调忽略规则很复杂：不是所有 generated file 都不应该提交。
- 例子 1：`package-lock.json` 这类文件可能就需要提交。
- 例子 2：你可能生成一个 header；这个 header 也可以提交，甚至提交后方便编译。别人不需要下载很多依赖，就能编译项目；即使他们那里没有生成 header 的工具链，也能编译。
- 讲者结论：没有绝对规则。

## 软件工程 best practice 检查：规则化工具 vs 大模型的耐心与情绪价值

---

- 讲者自述做软件工程研究；一个很标准的研究是：研究 Git 项目是否满足 best practices，不满足就给人一个 warning。
- 像 `.DS_Store` 这种东西需要一条规则；实际会有好多好多规则，最后做一个小工具，在 `git commit` 之前告诉你 OK 不 OK。
- 讲者认为，这些东西“都被大模型杀死了”：大模型可以非常耐心地逐项检查，还会给你 emoji，让你开心、缓和情绪。

## 导师纠正学生自带压力：AI 可能替代一部分“老板批评”的负情绪

---

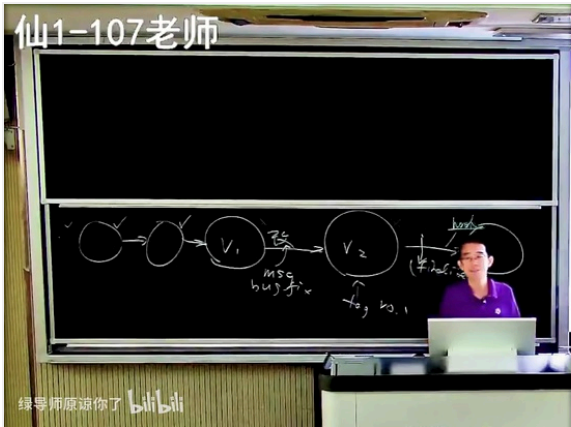
- 讲者以导师身份举例：把学生叫到办公室。第一，自己会高血压；第二，还得装作 nice。
- 他会说：“你提交的这个项目不太符合软件工程实践……”。
- 学生听的时候会觉得导师不满意；不管导师情绪多正常，只是在纠正、教知识，学生总是有压力：“老板批评我了”“我做了一件不该做的事”“这我 supposed 应该知道”。
- 讲者认为 AI 真的解放了学生，使他们不必面对那种“你无法控制的力量”。（半开玩笑）

# Hook：把自动化动作挂到提交或 agent 流程的某个事件上

- 讲者还说 AI 很强；最近在一个更严肃的项目里，希望把每个细节做好，于是谈到 Git 提交前的 hook 机制。
- Hook 可以挂到程序函数调用：函数调用结束时干一件事；也可以加在 agent 里：某件事发生时干一件事。
- 讲者自己的例子：一个 hook 会在 agent 完成、需要我输入时播放“叮”一声。

## 第 19 部分

仙1-107老师



绿导师原谅你了 bilibili

### 大写还是小写？

**真实项目中的一个选择**

```
[TYPO-CHECK] Docs should be typo-free
v.s.
[typo-check] Docs should be typo-free
```

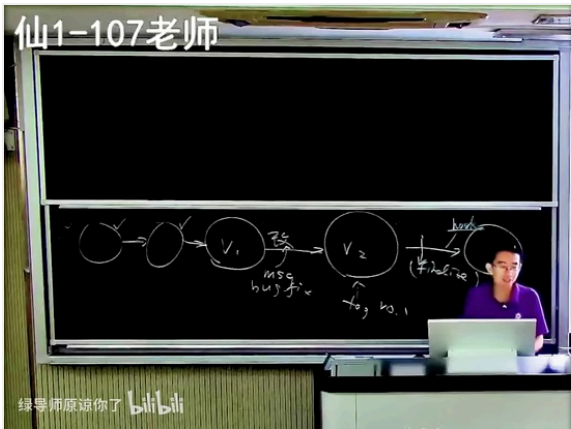
Check-rule 的名字应该是大写还是小写？

**直觉也值得追问**

我的直觉是“大写似乎更好，但不是极强”，但不太说得原因。

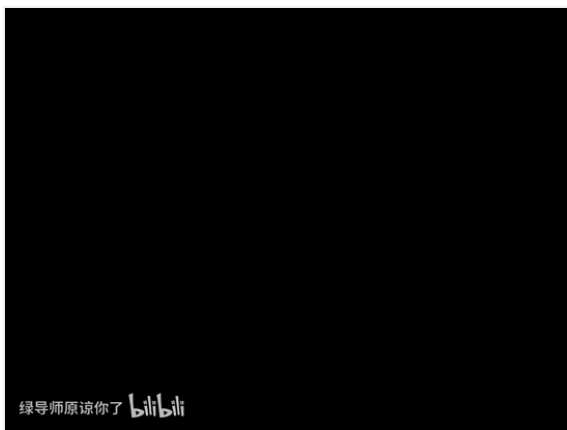
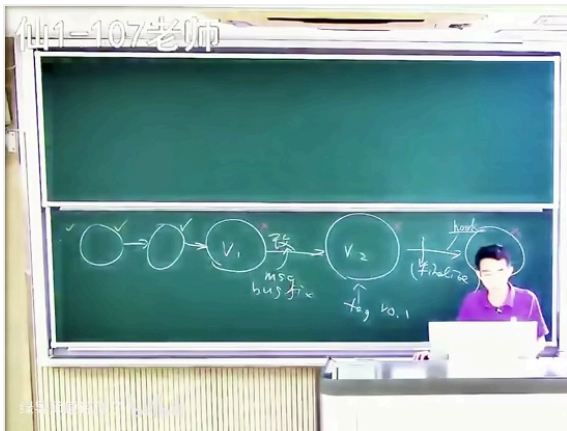
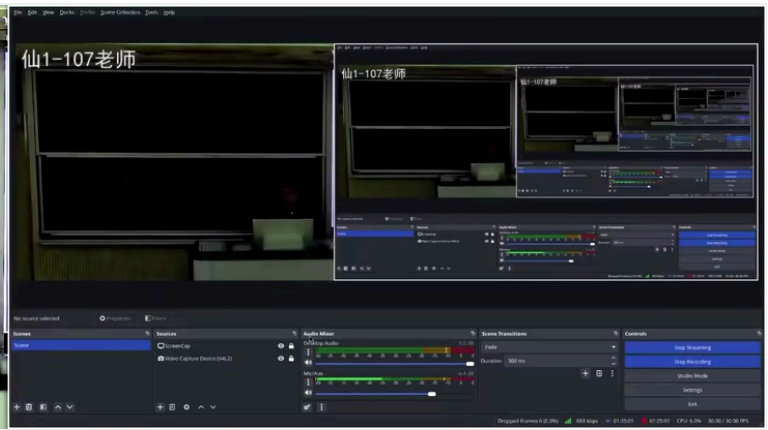
✦ 先想一想：方括号里的字符，是一句话的一部分，还是一个可以被引用的规则名字？

仙1-107老师



绿导师原谅你了 bilibili

```
$ eza --git --group-directories-first --icons=auto
AGENTS.md  hello  hello.c
$ git add .
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .DS_Store
    new file:   AGENTS.md
    modified:   hello.c
$
```



## Hook 的价值：事件发生时自动触发动作

- 讲者用自己设的 hook 举例：agent 完成、需要我输入时，播放一声“叮”提醒。
- 也可以直接让 AI 配置全局 hook：“在 agent 需要输入的时候叮一声”。
- 讲者称 Claude Code（转录为 cloud code）、Codex，以及转写为“派他”的工具，都支持这种 hook。
- 同一思路可放到 Git pre-commit hook：在提交前做检查；讲者认为这个项目的工程标准更高。

- 现场演示：让 agent 处理声音 hook；问“有声音吗？”没有；后来听到很轻一声；再试又没有。讲者说 agent 已意识到他的意图，可以让它修；但讲者选择不让它修。

## 命名大小写是两难：两个都能用，但全大写为何感觉更强？

- 场景：给各种规则命名，可能有几十个 **checking rules**；选大写还是小写？
- 从使用角度，两者都没问题；只要没有 `title`、没有 `type`，就 OK（讲者原话，具体指代未展开）。
- 这类两难在学习和生活中无穷多；讲者主张：每个问题都多问一句，就会有很多长进和知识。
- 他让 GPT 解释；AI 的解释（转录中作“style check”）指出这是固定规则的标识，并引用 RFC，还分析了讲者为何觉得全大写更强。
- 对学习 Git 的建议：多问几句，把每个细节做好，会成为 **A lot better person**。

## Git 的底层入口：它是持久化数据结构，持久化的是目录树

- 有了 AGENTS.md、CLAUDE.md 后，下一步学 Git 如何工作、底层原理。
- 讲者推荐去看他在操作系统课里讲 Git 的那一段；这里只讲开头。
- 核心主张：**Git 是一个持久化的数据结构**；所以它首先就是一个数据结构。
- 它持久化什么？**目录树**。

逻辑链（讲者给出的思路）：

1. Git 管理不停产生的快照版本：**V1、V2、V3、V4.....**
2. 每个版本要保存一棵目录树。
3. 例如一棵树可以是：

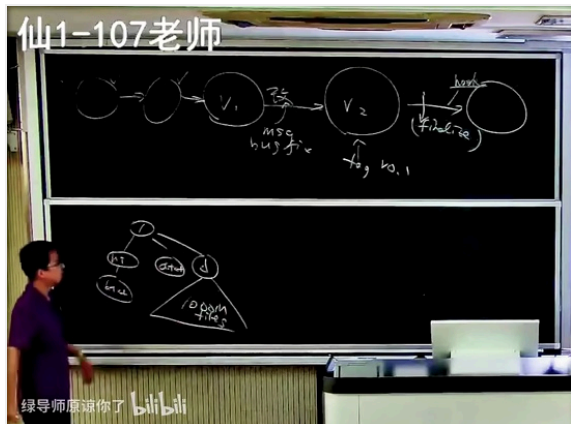
```
/
├─ hi
├─ A.txt
├─ B.txt
└─ D/
    └─ ... (很多文件)
```

4. 项目树可能非常大。

5. 若当前这棵大树是 V1，现在要创建 V2。

- 问题被引出：已有 V1 这棵大目录树，V2 如何创建？

## 第 20 部分

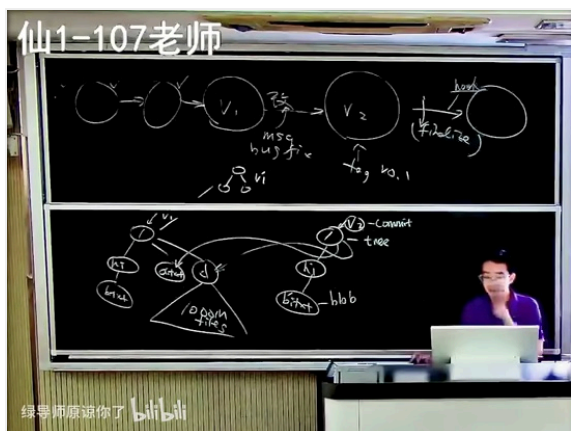
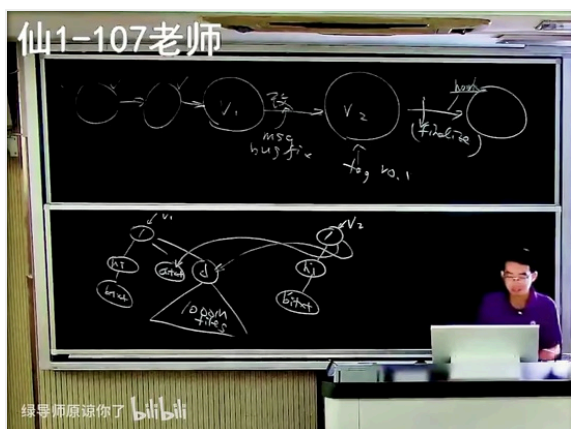


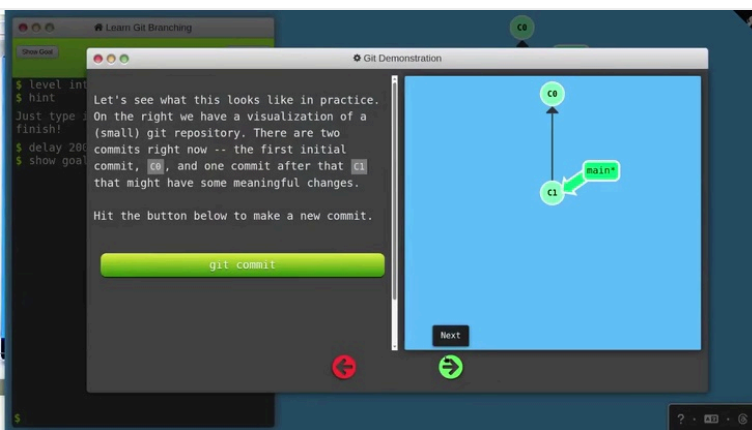
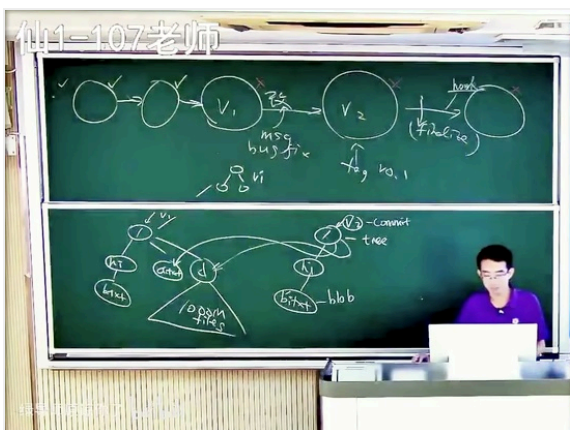
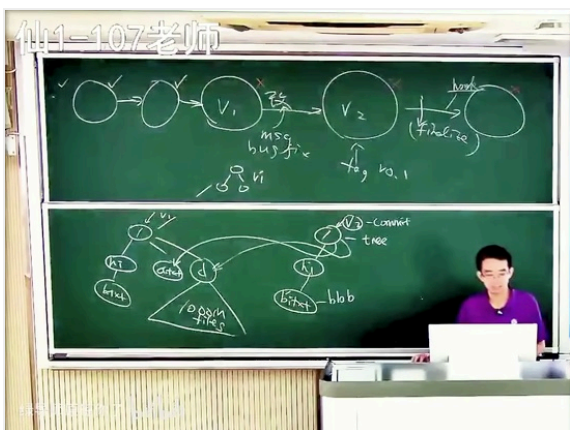
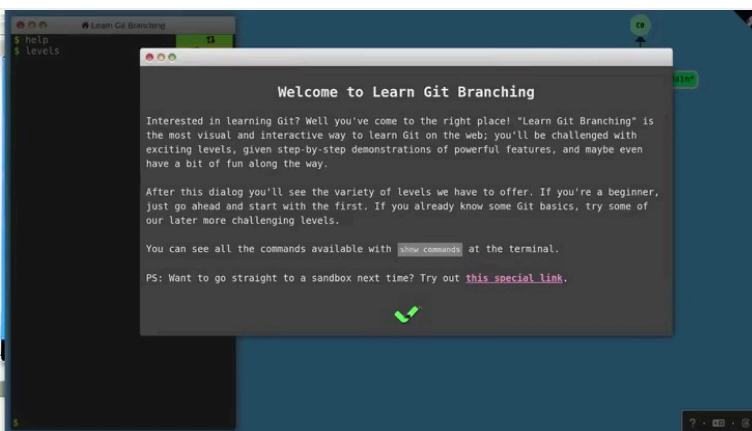
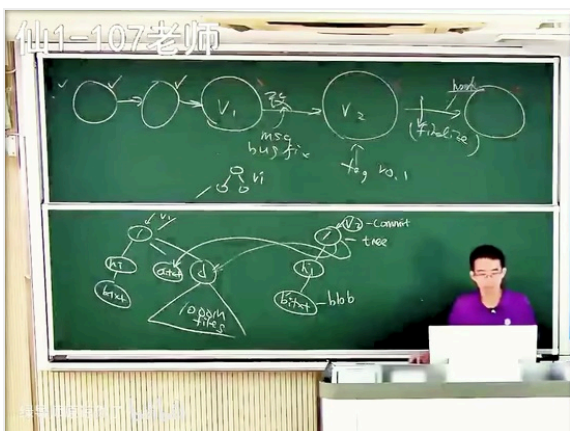
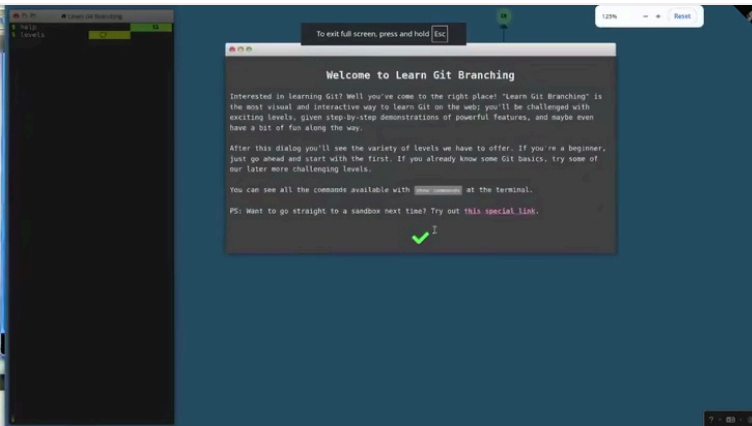
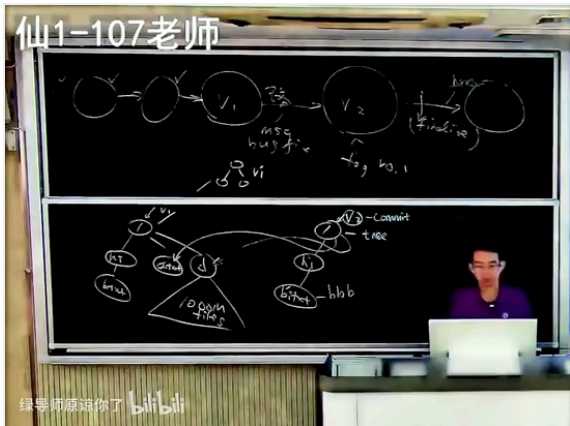
### 告别古法学习

计算机科学定律：你想要的就一定有人做

- 把关键细节以直观的方式呈现出来
- 人类的视觉系统是非常高效的
  - 软件可视化已经几十年进展缓慢，然后，忽然所有问题都解决了

1/3





## 改一个文件时，快照只需复制“到根的路径”

---

讲者先设问：项目已有 V1，现在要创建 V2；如果只改了 `b.txt`，是否要把所有文件都复制一份？答案是不需要。

逻辑链：

1. 改过的 `b.txt` 需要一个新版本。
2. 从该文件到根的路径都要复制一份：如果根或父目录仍指向旧节点，就等于还在引用旧的 `b.txt`。
3. 路径之外未变的 `a.txt`、`d` 等，可以用指针指向旧对象。
4. 因此 V2 只是一个小增量，而不是整棵树的副本。

讲者用 `a.txt`、`d` 的指针作示意：结构“虽然不太好看，但这就是 V2”。

## 目录深度有限，使“小增量”在超大项目中也成立

---

- 即使项目里有 100 万个文件，目录本身也不可能很深；讲者说没人会建 1000 层深的目录。
- 从渐进式披露角度也没必要建巨深目录；每一级索引可以指数级增加，很快就能把所有类分完。
- 所以只需要创建一个非常小的增量，就能得到一个版本的快照。

## Git 底层：commit / tree / blob 三种对象

---

- `.git` 里有各种 `objects`，它们是普通文件，带 `hash` 索引。
- 对象可看作节点：目录节点、文件节点、commit 节点。
- 三大类型：`commit object`、`tree object`、`blob object`。
- `blob` 是 `binary`，即字节序列。
- 这个数据结构可以不断向后生长；讲者认为版本记录问题由此解决。
- 讲者认为，从 `first principle` 出发，一个大学生完全有可能设计出这样的东西。

## 纸面模型不够踏实，就用可视化 Git 模拟器验证

- 讲者预想听众会问：真实 Git 真是这样实现吗？老师说的 commit object、tree object、blob object 真有吗？
- 如果发生分叉：可以从 V1 另建一棵树，例如 V1'，世界就分叉了；接着自然会问 merge 到底会发生什么。
- 讲者引用“计算机科学定律”：你想要的东西一定有人做过。
- 他打开浏览器，使用以前上课也用过的可视化工具，评价它是“古法编程”的巅峰之作之一。
- 该工具能可视化展示 Git 如何工作，并模拟各种命令，如 `git status`。
- 演示中已有修改；执行 `git commit` 后，右侧动画显示多出了一个版本。

### 第 21 部分

仙1-107老师

Learn Git Branching

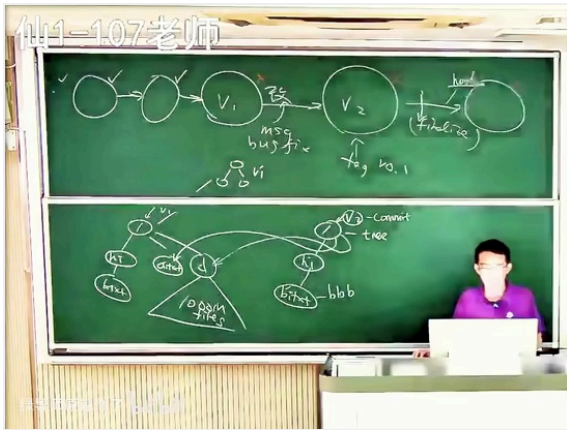
Show Goal ↑ Level Introduction to Git Commits Instructions

```
$ level intro1
$ hint
Just type in 'git commit' twice to finish!
$ delay 2000
$ show goal
```

On branch main  
# Changes to be committed:  
# modified: cal/OskiCostume.stl  
#  
# Ready to commit! (as always in this demo)  
\$ git add .  
There is no concept of adding / staging files, so that option or command is invalid!  
\$ git commit  
\$ git checkout C1  
\$ git checkout C1  
\$ git checkout C2  
\$ git checkout C0  
\$ git checkout C1  
\$ git checkout C2  
\$ git checkout C1

Leave site?  
Changes you made may not be saved.  
Cancel Leave

C0  
C1  
C2  
main  
HEAD



```

68 91ac2bf64805995a5c6eae10d8ae646f821ae0
6c a7227b3ac8ee772063493d8e96ae19b3c38c9c
82 c557075371c05f24b5d07ef0cb46bad14a9557
99 2137f0084d075e12bcaa3f1f476ff48522c2ad
9c e44b0f4fa8412d4fe2c35d13c40638f39b2682
a2 27ea44568c9128bd831436bcdfcd42c9b5353
a9 a843e5744b2df27a7ece378b5d64557cd867b0
b2 f997621bea95c08664e45bcbdc81f3c264a051
bc cfe440e93b79e963ff968a6829649eade74f3d
e6 9de29bb2d1d6434b8b29ae775ad8c2e48c5391
e9 2569d01017c5a4cbad8c797325df200ef8c096

info
pack
refs
├── heads
│   └── main
└── tags
27 directories, 37 files
$

```

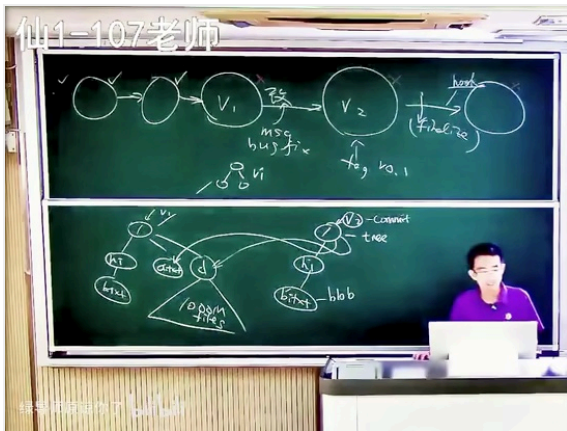
```

$ cd /tmp/hello/
$ git blame
usage: git blame [<options>] [<rev>] [--] <file>

<rev> options are documented in git-rev-list(1)

--no-incremental show blame entries as we find them, incrementally
-b do not show object names of boundary commits (Default: off)
--no-root do not treat root commits as boundaries (Default: off)
--no-show-stats show work cost statistics
--no-progress force progress reporting
--no-score-debug show output score for blame entries
-f, --no-show-name show original filename (Default: auto)
-n, --no-show-number show original linenumbers (Default: off)
-p, --no-porcelain show in a format designed for machine consumption
--no-line-porcelain show porcelain format with per-line commit information
$

```



### 把真实的 Git Repo 展开给你看

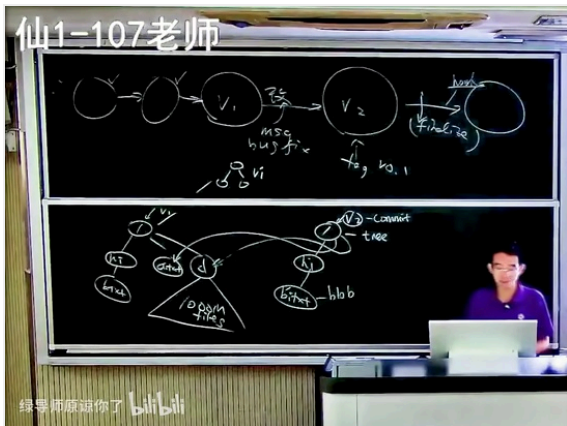
“实现 gitv 命令行工具 (nodejs)，把一个**真实**的 git repo (你可以使用 git 命令) 路径里的主要概念彻底、完整地可视化。展示任何对象之前，都从文件系统里的真实文件、原始 bytes 出发，展示一步步如何解析，解析成了什么。通过空间合理的方式排布各类对象，并支持拖动：明快、极简的风格，不要多余的 tooltip text 等，在能够 self-explain 的时候绝不出现文字。大小文件的可视化都要有不错的视觉效果，对于有具体内容的文件，有适当的大小和视觉风格显示内容、diff 等。启动一个 http server，持续监听目录中 git 的变化，并且在发生变化后，用 css 动画效果展示 repo 发生的变化。浅色背景适合课堂展示。

**直接生成，现场对比**

- GPT-6 直出 v.s. <https://learnitbranching.js.org/>; 国模直出 v.s. <https://code.cs61a.org/>

**从看懂到学懂**

- 算法可视化研究的启发：学习者怎样使用可视化，比可视化展示了什么更影响学习效果。Hundhausen 等, 2002



```

$ eza --git --group-directories-first --icons=auto
AGENTS.md  hello  hello.c
$

```

讲者判断：今天可以直接 web coding 出 Git 可视化工具，但质量未完美

- 讲者先回顾之前看到的工具：它甚至模拟了各种命令，如 `git status`、`git commit`、`git checkout`；点击版本可看到版本变化，用这种方式教 Git 如何工作。
- 讲者认为到今天可以直接用 web coding 做这些了；不过他也承认 web coding 的质量还并没有那么好。
- 他自己直接 web coding 了一个可视化整个 Git repo 的工具。工具里甚至还能看到一些未提交的代码。
- 讲者评价模型 GP6（按转录）虽然有视觉能力，但一次直出仍产生了一些 slop。

## 提示词把目标钉在“真实 .git 文件”而不是模拟

- 提示词要求：实现一个 **Git 可视化命令行工具**；针对一个真实的 Git repo，可以使用 Git 命令；把里面的主要概念彻底、完整地可视化展示。
- 关键约束：任何对象都从**文件系统里的真实文件**出发；这和 **Learn Git Branching** ([learngitbranching.js.org](https://learngitbranching.js.org)) 不一样——那个是模拟出来的，建立了一个 Git 模型；本工具要在 **live 的真实 Git repo** 上直接看到发生了什么。
- 要求从真实文件、原始数据出发，展示**一步一步如何解析、解析成了什么**。
- 还要求工具持续监听目录中 **Git 的变化**，变化后用**动画效果**展示 repo 发生的变化。

## 逻辑链

1. Learn Git Branching 用模拟 Git 模型教学 → 学生看到的是被建模的命令与版本变化。
2. 讲者要求新工具从真实 `.git` 文件出发 → 看到真实对象和解析过程。
3. 工具持续监听 `.git` 变化 → 真实操作可触发动画展示。
4. 因此演示中可以删除文件、查看提交，并观察对象、树、指针如何变化。**(整理者串联：把“从真实文件出发”与“监听变化”连成可现场观察 live repo 的工具。)**

## 演示一：删除文件，界面同步变化

- 讲者先把某文件删掉；右边文件随即没有了。
- **(整理者串联)** 这一步用来证明工具确实在跟随真实 repo / 工作区变化，而不是只展示静态模拟。

## 演示二：main 分支的树与最早提交

- 讲者查看这棵树：这是 **main 分支**，就像他画的图一样。
- 最早的一次提交是 **“I did some cool stuffs”**。
- 提交里有一个空的 `hello.c`。
- 这个空的 `hello.c` 文件来自 `.git/objects` 下的一个 object，它真的是一个 **blob object**。
- 工具显示它的**字节序列**，然后帮助解压缩。
- 讲者指出：对很小的文件来说，压缩反而是浪费——**31 字节解压出 23 字节**；但这不要紧，因为它是一个 blob。
- 看到 **blob 15** 文本：这是一个 blob object，大小为 15，内容含 `int main` 等。
- 除了这种 blob，还有 `hello` 的二进制文件：**blob 158**，开头是可执行文件头（转录为“7FERF”）；讲者说也能看到它。

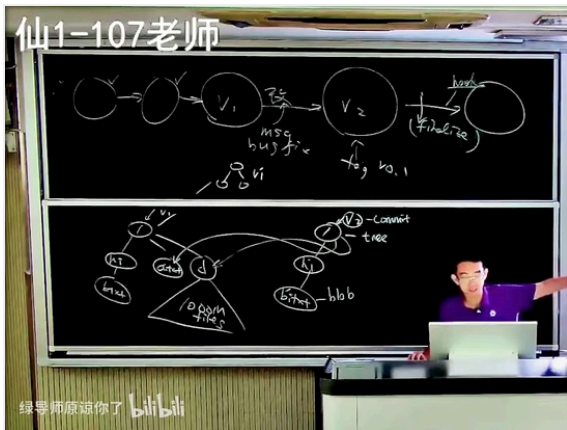
### 演示三：tree object、权限、文件名与指针

- 还有 **tree object**；同样能看到它磁盘上面的原始数据：52 字节解压出 43 字节。
- 这个 tree 的内容显示：644 代表文件的访问权限；hello.c 真的是 blob；解压完后以文本形式存储到 hello.c。
- 树结构里有 **blob object**：mode 644、name hello.c；还有 **tree object**。
- 在这个版本的根 tree 中，讲者指出有两个文件：一个 755、一个 644；其中一个 hello 是可执行的，另一个是 hello.c。
- 指针链接可以拖动查看：tree 指向 blob；讲者把它拖过来，一条链接指向这里，另一条指向那里。

小结：讲者用 live repo 的 .git 对象演示，把 Git 从“命令模拟”推进到“逐字节观察对象如何被解析、引用和变化”；同时他承认当前 web coding 一次直出仍有 slop。

## 第 22 部分

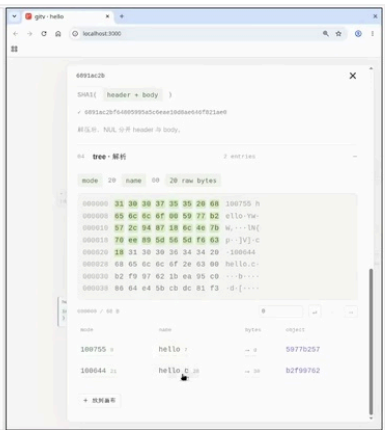
仙1-107老师



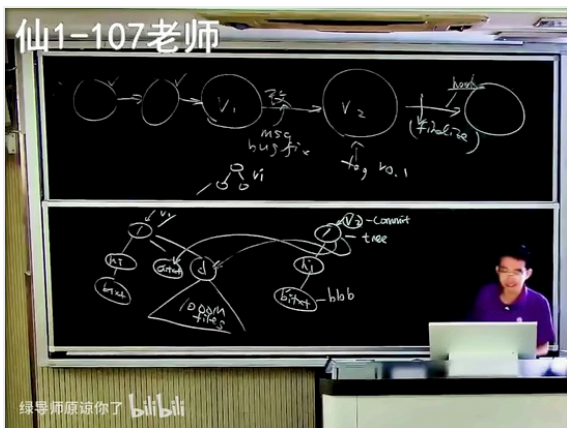
绿导师原谅你了 bilibili

```
$ exa --git --group-directories-first --icons-auto
= AGENTS.md  hello @ hello.c
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .DS_Store
    new file:   AGENTS.md
    modified:   hello.c

$ git reset --hard
HEAD is now at 196a2a9 chore: ignore compiled hello binary
$
```



仙1-107老师



绿导师原谅你了 bilibili

### 把真实的 Git Repo 展开给你看

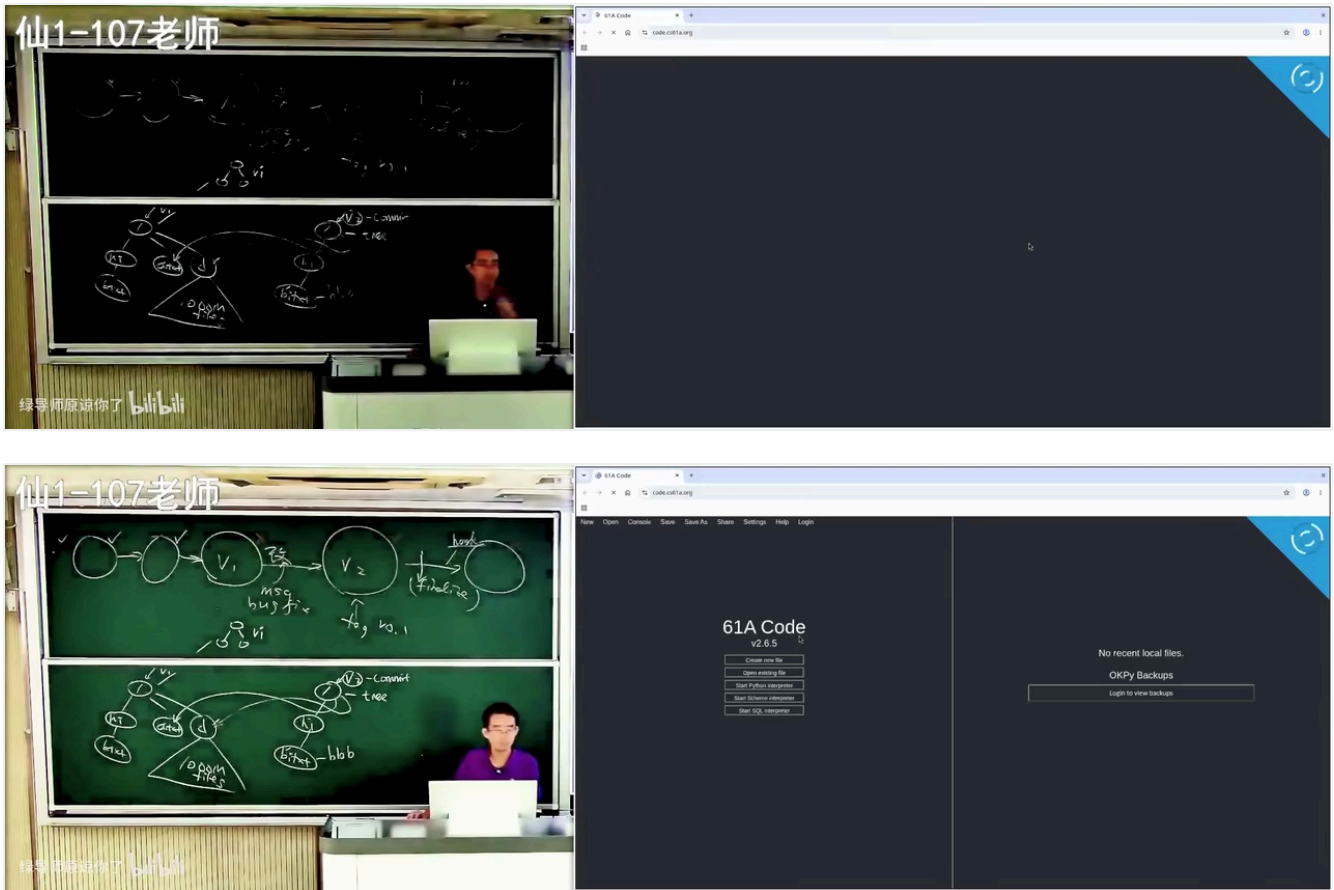
“实现 gitv 命令行工具 (nodejs)，把一个**真实**的 git repo (你可以使用 git 命令) 路径里的主要概念彻底、完整地可视化。展示任何对象之前，都从文件系统里的真实文件、原始 bytes 出发，展示一步步如何解析，解析成了什么。通过空间合理的方式排布各类对象，并支持拖动：明快、极简的风格，不要多余的 tooltip text 等，在能够 self-explain 的时候绝不出现文字。大小文件的可视化都要有不错的视觉效果，对于有具体内容的文件，有适当的大小和视觉风格显示内容、diff 等。启动一个 http server，持续监听目录中 git 的变化，并且在发生变化后，用 css 动画效果展示 repo 发生的变化。浅色背景适合课堂展示。”

直接生成，现场对比

- GPT-6 直出 v.s. <https://learngitbranching.js.org/>；国模直出 v.s. <https://code.cs61a.org/>

从看懂到学懂

- ◆ 算法可视化研究的启发：学习者怎样使用可视化，比可视化展示了什么更影响学习效果。Hundhausen
- 等，2002



## Git 可视化 demo：把“可执行 Hello”和指针关系拖到图上

- 讲者展示一个 `hello.c`（原转写：hello点i see）的 Hello 是可执行的；然后把指针链接拖来拖去，让一个链接指向这里、另一个指向那里。
- 这套视觉效果不完美：讲者说自己没有调，是 **one-shot** 直出；只通过第二个 prompt 让它去掉了一点东西，所有问题都没修。
- 讲者预计，再和 AI 对话几轮，应该就能得到近乎完美的 **Git visualization**。
- （整理者串联）这里把“程序可执行”与“Git 对象/引用可视化”并置，是为了让后续分支、提交、冲突都能落到可见的指针和对象图上。

## 创建分支：分支 = refs/heads 下的指针；HEAD = 指向指针的指针

- 讲者让工具创建两个分支，动画直接展示效果，左右实时同步。
- 看到的東西是 **refs/heads/A**（原转写：a rapes，是 heads 下的 A）。讲者解释：分支对应 **refs** 里的文件/引用，它本质就是指针；所谓分支就是指向这些东西的指针。

- **HEAD 是指向指针的指针**。在磁盘上就是这些字节；这些字节/对象再指向 **commit object**（原转写：tout）和 **tree object**（原转写：trial）。
- 看分支 A：A 是一棵树，树里面有目录/条目，包含新的 **hello.c**，内容是 **int main(void)**。B 类似，指向另一个修改过的 **hello.c** 版本。
- 视觉效果仍不好，但讲者相信多轮对话后能搞定，得到近乎完美的 Git 可视化。
- 讲者给出的链条：分支 → refs/heads 下的引用/文件 → 指针；HEAD → 当前分支指针；磁盘上表现为字节和对象指向关系。

## Merge conflict：两人重构同一处 → index 里保留 base/ours/theirs，工作区显示冲突标记

---

- 讲者让工具展示 **merge conflict** 状态。
- 场景：两个开发者在开发，一个人做了修改；你和团队成员都对同一段代码不爽，两个人都重构，于是进入冲突。A 在一边，B 在另一边；把 A 拉到本地时，就会看到冲突状态。
- 冲突现场：hello 的内容变成冲突展示；未提交区域里有几个版本；index 中 **hello.c** 背后留着三个版本：**base**、**ours**、**theirs**（原转写：贝斯、hours、theirs）。这些是工作区看不到、但 Git 知道的。
- 工作区看到的 **hello.c** 是带尖括号的冲突内容。
- 让 AI resolve 后：临时文件消失；merge 创建了一个 **commit object**，它的 **tree** 是解决后的树；这个 commit 有 **两个 parent**（原转写：两个泰伦），还有 **author**（原转写：offer）。
- 讲者指出，指向关系被正确保留；**tree** 中选的是这个解决后的版本（原转写：TRIOFF）。他不再继续展示。
- 逻辑链条：两人修改同一处 → 合并产生冲突 → Git 在 index 保留 base/ours/theirs → 工作区显示冲突标记 → resolve 后生成含两个 parent 的 merge commit，指针关系保留。

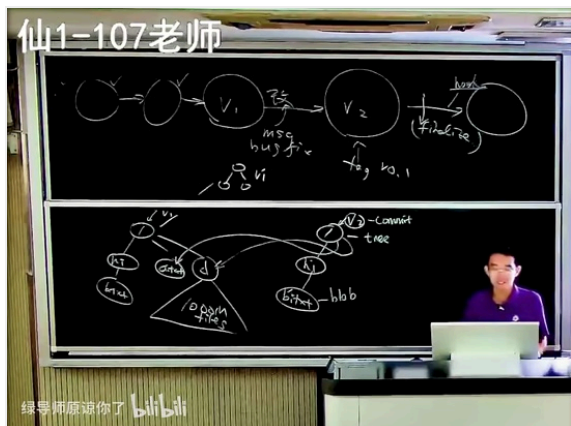
## 讲者观点：vibe coding 适合有时间的人边调边学

---

- 讲者认为 **vibe coding** 很好用（原转写：ww coding / web code）。
- 他说，像你们这个年纪、尤其有时间的时候，最大的乐趣就是 **vibe coding**；vibe code 完了以后各种调，然后跟着它学习，知识会爆炸式增长。

- 再举 CS 课程例子：上 SICP 时，有没有用过官方的 [code.cs61a.org](https://code.cs61a.org)（原转写：code点cs6161A打到）；讲者问大家用过吗。

## 第 23 部分



### 把真实的 Git Repo 展开给你看

“实现 gitv 命令行工具 (nodejs)，把一个**真实**的 git repo (你可以使用 git 命令) 路径里的主要概念彻底、完整地可视化。展示任何对象之前，都从文件系统里的真实文件、原始 bytes 出发，展示一步步如何解析，解析成了什么。通过空间合理的方式排布各类对象，并支持拖动；明快、极简的风格，不要多余的 tooltip text 等，在能够 self-explain 的时候绝不出现文字。大小文件的可视化都要有不错的视觉效果，对于有具体内容的文件，有适当的大小和视觉风格显示内容、diff 等。启动一个 http server，持续监听目录中 git 的变化，并且在发生变化后，用 css 动画效果展示 repo 发生的变化。浅色背景适合课堂展示。”

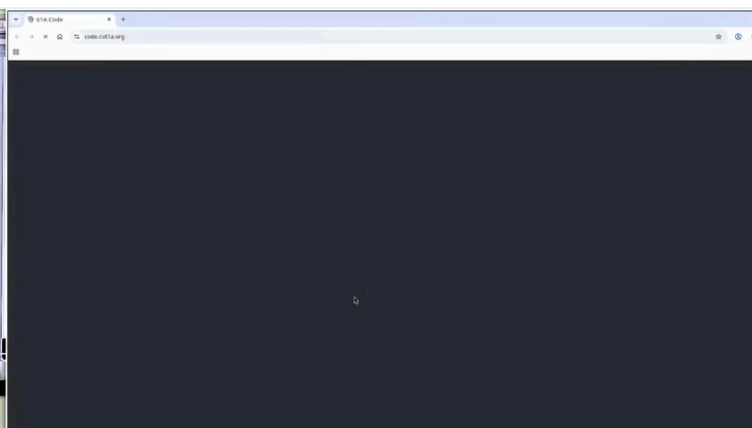
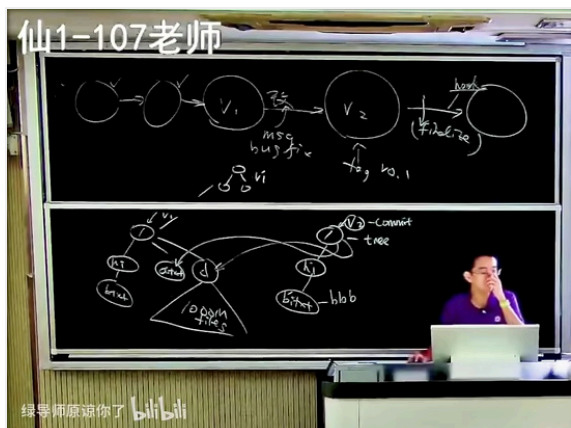
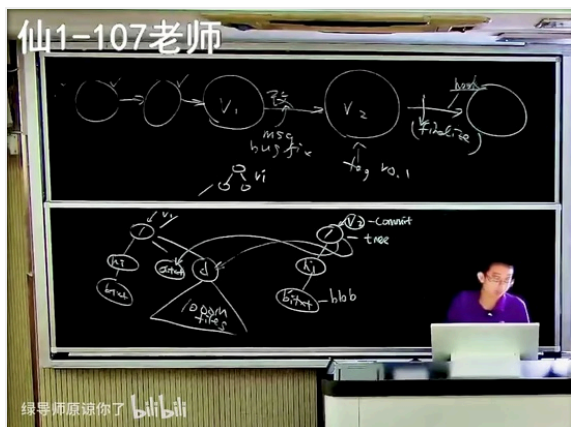
### 直接生成，现场对比

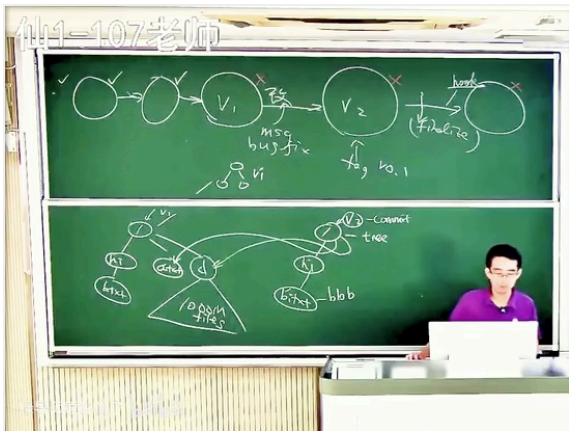
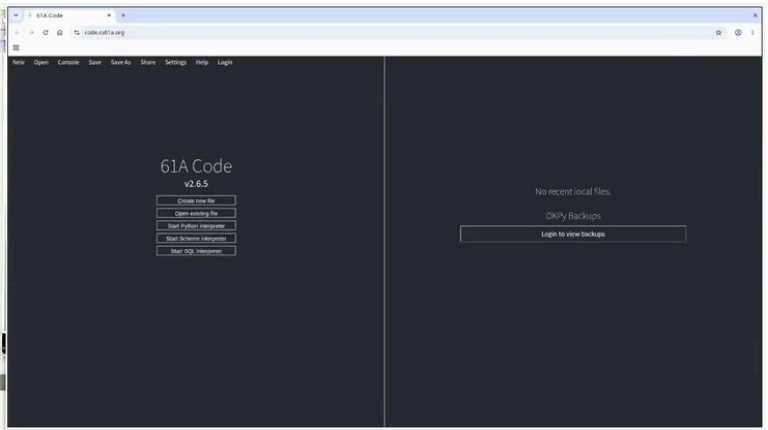
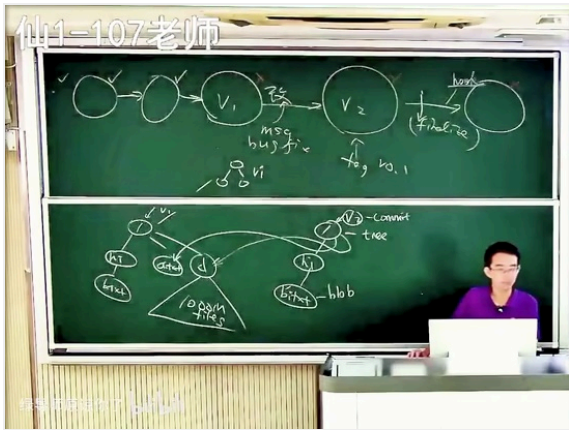
- GPT-6 直出 v.s. <https://learnitbranching.js.org/>；国模直出 v.s. <https://code.cs61a.org/>

### 从看懂到学懂

- ✦ 算法可视化研究的启发：学习者怎样使用可视化，比可视化展示了什么更影响学习效果。Hundhausen
- 等，2002

2/3





## 把真实的 Git Repo 展开给你看

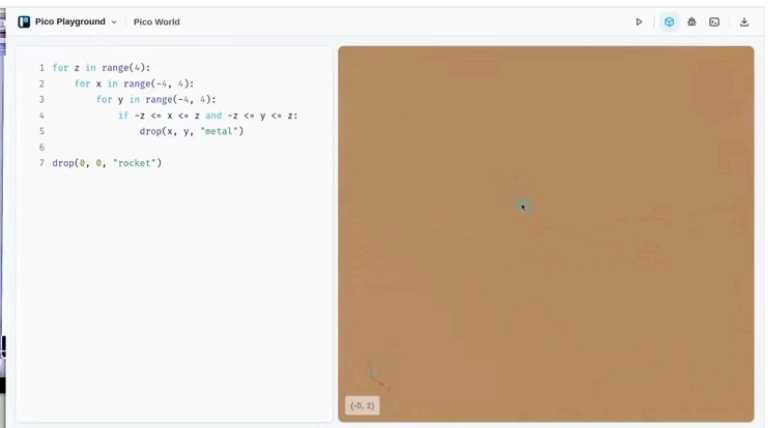
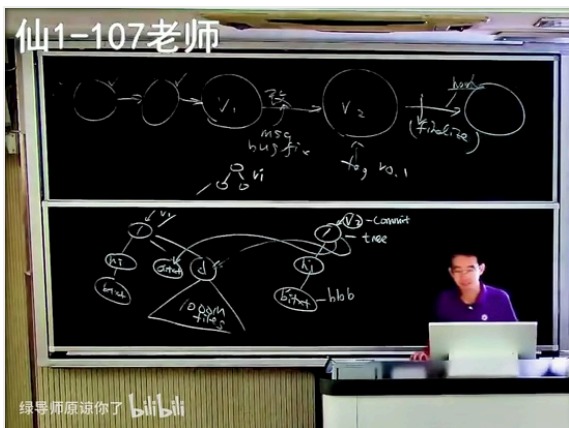
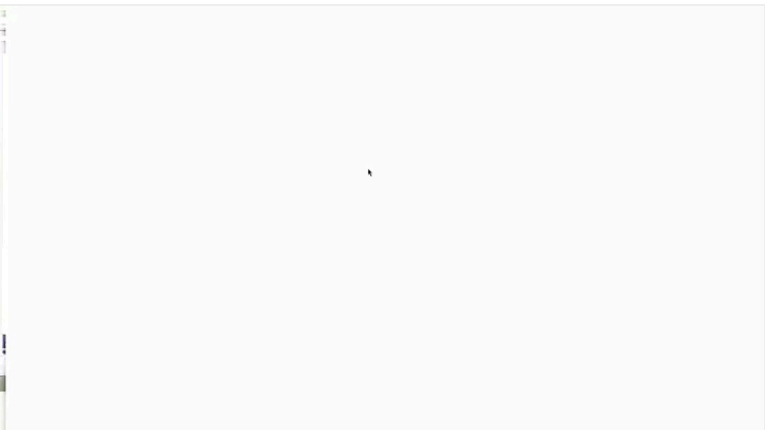
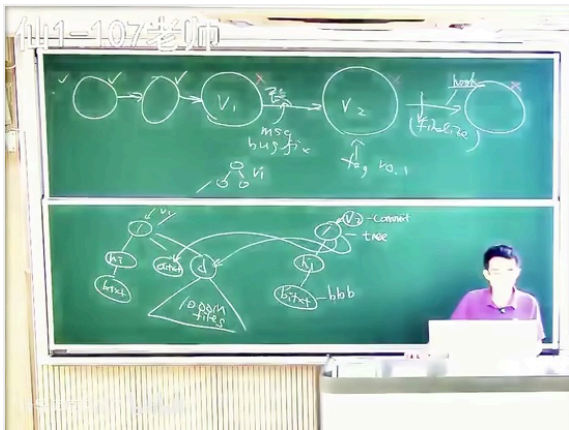
“实现 gitv 命令行工具 (nodejs)，把一个**真实**的 git repo (你可以使用 git 命令) 路径里的主要概念彻底、完整地可视化。展示任何对象之前，都从文件系统里的真实文件、原始 bytes 出发，展示一步步如何解析，解析成了什么。通过空间合理的方式排布各类对象，并支持拖动；明快、极简的风格，不要多余的 tooltip text 等，在能够 self-explain 的时候绝不出现文字。大小文件的可视化都要有不错的视觉效果，对于有具体内容的文件，有适当的大小和视觉风格显示内容、diff 等。启动一个 http server，持续监听目录中 git 的变化，并且在发生变化后，用 css 动画效果展示 repo 发生的变化。浅色背景适合课堂展示。”

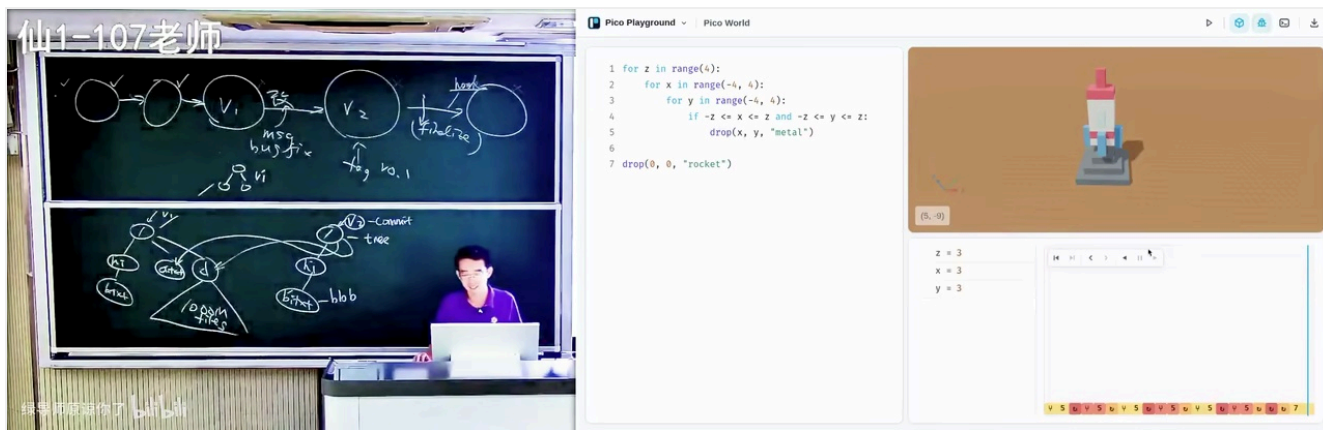
### 直接生成，现场对比

- GPT-6 直出 v.s. <https://learngitbranching.js.org/>; 国模直出 v.s. <https://code.cs61a.org/>

### 从看懂到学懂

- ◆ 算法可视化研究的启发：学习者怎样使用可视化，比可视化展示了什么更影响学习效果。[Hundhausen 等, 2002](#)





## 论证主线：AI 生成的代码可以有用，前提是架构由人把关、场景匹配教学

1. 浏览器里的官方 Python IDE 已能承担课程作业。
2. 讲者的小项目让 AI 一句话生成大量 **slop**（转录原词），但自己把控架构。
3. 于是系统可做 Web 教学应用，也可打包成 **Electron/EXE**；还能可视化程序。
4. 面向三、四年级，第一课必须避免 **PyCharm** 式复杂界面，先让孩子觉得 Python 好玩。
5. 汉诺塔调试器把递归过程可视化，减轻上课负担。
6. 讲者认为真正好玩的是给小孩一个 **API** 和一个世界，并让他们可调试。

（整理者串联：这条链的重点不是“AI 能写代码”，而是“人把关架构 + 极简交互 + 可视化/可调试”共同构成可用教学工具。）

## 官方 Web IDE 证明浏览器里可跑完整 Python 教学环境

- 讲者问做 CS、上 SICP/CS61A 的学生是否用过官方 **code.cs61a.org**（转录作“code点cs6161A打到”）。
- 这是一个 **Python IDE**，放大后也能用，是如假包换的 Python IDE。
- 可运行；可能正在把 **Python runtime** 加载到本地；讲者这里访问不算快。
- 基本是完整功能；需要登录；可以完成作业。
- （整理者串联）这个参照说明：浏览器里的教学 IDE 已足以支撑课程作业。

## 小项目：AI 一句话生成 slop，但架构由讲者把关

---

- 最近因为一个小项目，讲者被迫从外部搞到一个类似的东西。
- 里面有很多 **slop**（转录原词），都是讲者直接让 AI 一句话生成的。
- 讲者判断：它可能不是 IDE 级别，但效果不错。
- 关键做法：讲者在架构设计上稍微把控、把了关；代码质量则不太管控。
- 结果：
  - 可作为网页应用，教大家编程；
  - 也可作为 **Electron** 应用，像 VS Code 一样一个 EXE 双击打开；
  - 可可视化任何程序。
- （整理者串联）讲者的逻辑是：AI 负责生成，人负责架构把关；这样产出的工具即便代码质量参差，也能承担特定教学任务。

## 教三、四年级 Python：第一课要制造“很好玩”，不能直接上 PyCharm

---

- 因为要给三、四年级小朋友教 Python，这是很有挑战的事。
- 讲者认为：第一节课如果直接教他们 **PyCharm**，就全完蛋了。
- 所以要做极简界面，整个都是 Web 的；大量由 AI 生成（转录作“AI is lap”）。
- 讲者说不太管控代码质量，但对架构有把控。
- 第一节课的目标：给小朋友一个印象——**Python 很好玩**。
- 讲者举的“好玩”例子：
  - 把整数变成字符串，这是学生熟悉的；
  - 把 **Unicode** 字符编码成一个整数，再 decode 回来；
  - 把 `hello` 乘以十，打印十份。
- （整理者串联）这些例子不是讲语言系统，而是用即时可见的结果制造趣味，降低第一课门槛。

## 汉诺塔：调试器把递归移动过程变成可播放可视化

---

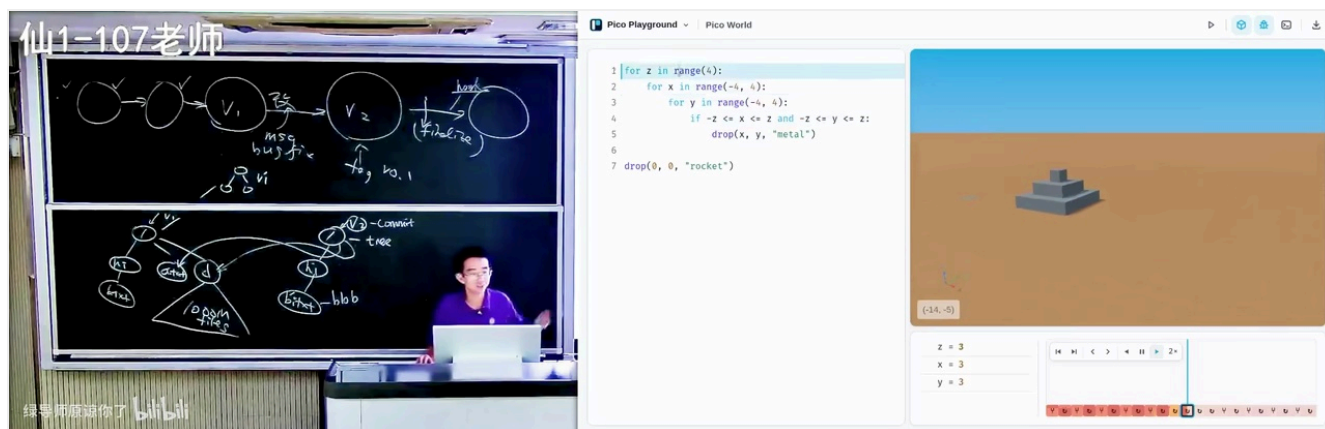
- 讲者展示汉诺塔时有 **debugger**。
- 在递归之前，调试器会只告诉他“应该怎么移动”。

- 可以往后播放，它就会真的移过去；可以换一个盘子更多的例子；也可以往前拖、往后拖。
- 讲者评价：看起来就是一个不错的可视化。
- 这个也是外部的、完全外部的。
- 效果：上课的负担轻多了。
- （整理者串联）这里承接前面的教学目标：把递归这种抽象过程变成可播放、可回退的动画，学生更容易看明白。

## 真正好玩：给小孩 API 和一个可调试的世界

- 讲者问：三、四年级的小朋友能编什么程序？
- 他的答案：给他一个 **API**，给他一个世界，他就可以玩起来。
- 同时讲者也可以调试他。
- 这是一个完全 **flat** 的程序。
- 例子：可以看程序运行过程；一个 **ZXY** 的循环（原文如此）；方块从上往下掉，每一次都是真实的。
- 讲者可以往前走、往后退；视觉效果他显然稍微调了一下。

## 第 24 部分





- 例子包括 **Fibonacci number / 斐波那契数列递归**；讲者回到开头播放 **插入排序** 如何一个一个比较。
- 讲者觉得“太容易了”。
- 他有一个 **skill**（原文提到“scale”）可以把它编译，把任何想要的东西编译成在模型能力下的产物。
- 还试了 **DeepSeek 版本**（原文“dsk flash division”，识别不确定）：讲者说它是国产的、便宜的模型，也可以出课件，效果已经达到完全可用。
- 接着他说：“然后我就可以去挣钱了。”

### 真正有价值的品质：愿花时间学习；细节标准会水涨船高

- 讲者认为，今天真正有价值的品质，是愿意花一点时间学习。
- 逻辑链：
  1. 一旦有人学会把细节做好，招聘单位就会用这个标准来衡量你。
  2. 所以“把细节做好”的 **bar** 会不断升高。
  3. 如果不成为能把细节做好的人，在 job market 上的竞争力就会下降。
- 对比案例：
  - 一个研究生被问 **git merge 之后会发生什么**，完全没概念。
  - 另一种人则在每一门专业课、每一个算法、每一个细节中，在 AI 指导下巩固概念、真正消化概念，并能从零开始构建这些概念。
  - 讲者认为这两种人在 job market 上的竞争力不同。
- 讲者还提出一种可能：AI 太强，所有人没有竞争力，也许这是最好的状态；否则危机感存在——世界上有很多“那样的人”，你如果不成为那样的人，上了 job market “你就死了”。

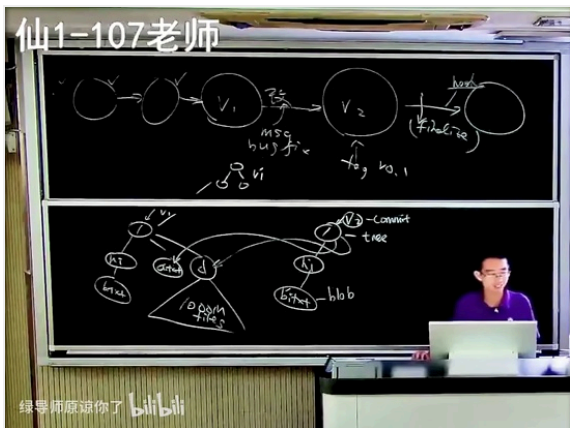
### “上课耽误学习”真实存在：AI 直出 vs 照本宣科

- 讲者认为“上课耽误学习”真实存在：很多老师真正好的东西“打开都没有打开过”。
- 那些真正好的代码、真正好的项目、符合工程规范的项目，老师只是对着课本讲一遍。
- 对比今天看到的 AI 直出东西：刚才整个框架，讲者大约用 **两天碎片时间** 就全部完成，包括视觉效果调整。
- 讲者强调：做这些事情已经没有任何成本。

小结：AI 让作品和课件制作近乎零成本，但市场会跟着抬高细节标准；讲者因此把“愿花时间学习并用 AI 真正消化概念”当作核心竞争力。

---

仙1-107老师



## “把细节做好”的 Bar

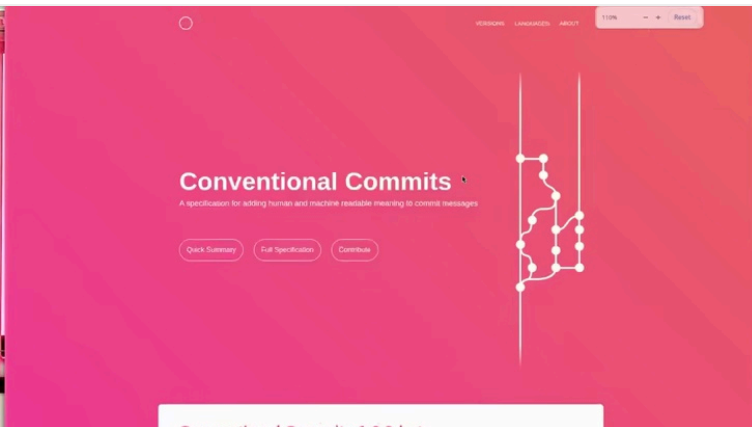
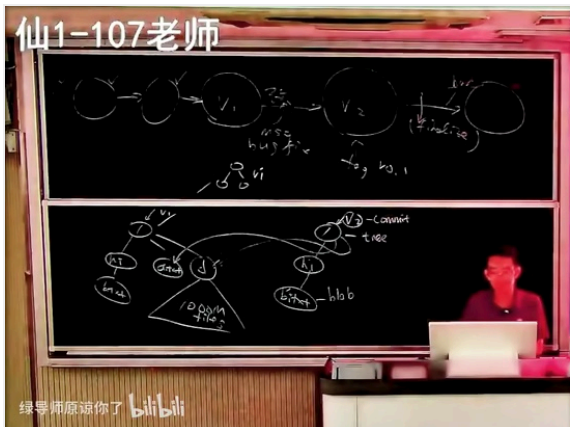
- 好消息：真正学懂和成绩是正相关的

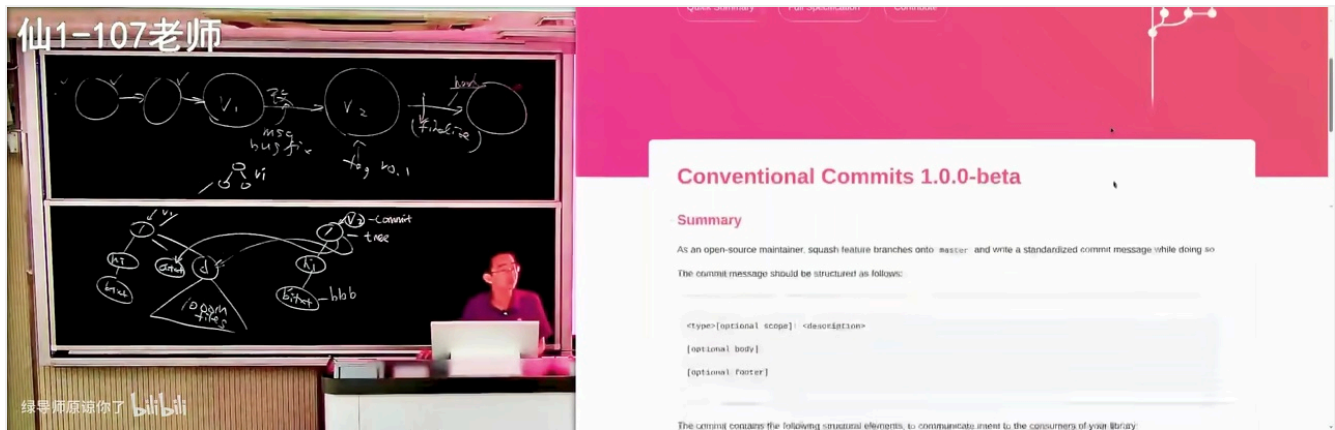
- **愿意学懂**，就成了真正有价值的品质。

遗憾的是，自己的学习时间非常有限



378





## 今天表面讲 Git，实质讲的是 best practice

- 讲者明确说：虽然今天好像在讲 Git，其实讲的是 **best practice**；这里面还有很多 best practice。
- 逻辑链（整理者串联）：AI 能在碎片时间低成本完成视觉调整等杂活 → 讲者感到自己“马上要淘汰”且学习时间有限 → 但 AI 的 commit message 仍会自由发挥、混中英文 → 因此需要 **Conventional Commits**、Markdown 中英文空格这类规范 → 讲者把这些工程细节的目的归为大项目中的一致性和可维护性 → 在 AI 时代这些可能不重要，但他仍建议回去读 best practice 文档、让 AI 辅助学习 → 关于及时 commit，复杂项目中修改越多越可能出错；小项目初期不要紧。

## AI 让实现成本趋近于零，讲者由此谈自己的时间焦虑

- 讲者说，自己用两天碎片时间就完成了全部工作，包括视觉效果调整，几乎没有成本。
- 他半开玩笑说，做这些事情已经没有成本，所以“我马上就要淘汰了”。
- 他补充：自己学习时间非常有限，很想学很多东西，但不像学生有那么多时间。

## AI 的 commit message：格式大体对，但仍会“自由发挥”

- 讲者让听众注意 AI 给出的 commit message。
- 前面的 commit message 都是英文：一行摘要 + 详细解释；有些还带 **scope**，说明“我干了什么”，再给总结和 **message body**。
- 但讲者认为 AI 还是挺 **slop**（草率）的：它开始自由发挥，混用英文和中文。
- 不过它大体仍遵循了 scope、总结、message body 这样的格式。

## 这类格式有标准：Conventional Commits

- 讲者指出，这种 commit message 格式是有标准的，今天可以找到 **Conventional Commits**。
- 它的 specification 很短，要求 commit 必须以 **type** 前缀开头；type 是一个名词，例如 `feature`（引入功能）、`fix`（修 bug）。
- 后面要有冒号，冒号后一定要有空格。
- 这些都是规范。

## 和同学合作时，文档格式规范也会变成“高血压时间”

- 讲者说，和同学合作时“高血压时间”比较多。
- 例如 Markdown 文档一般要求英文和中文之间加一个空格。
- 他不知道为什么大家开始用这个 convention，但好像大家都会用。
- 对他而言，看到“你好”和 `hello.c` 这种中英文混排该加空格，已经是肌肉记忆。
- 最让他难受的是同一个文档里有时加、有时不加：全篇一致，他会认为你在遵循某种 convention；不一致，他就头大。

## 工程细节的目的：大项目中的一致性和规范

- 讲者认为，这些工程细节都是为了工程师面对更大项目时，有一种一致性和规范。
- 例如代码要以同样方式排版，很自然地文档也要以同样方式排版。
- 这样人维护起来会更好。
- 他补充：在 AI 时代，这些可能好像也不重要，因为 AI 都会帮你解决。
- 所以他说“AI 我还是很喜欢的”；他认为所有用 AI 的学生都不会生成那样的 `slop`，至少不会生成那样的 `slop`。

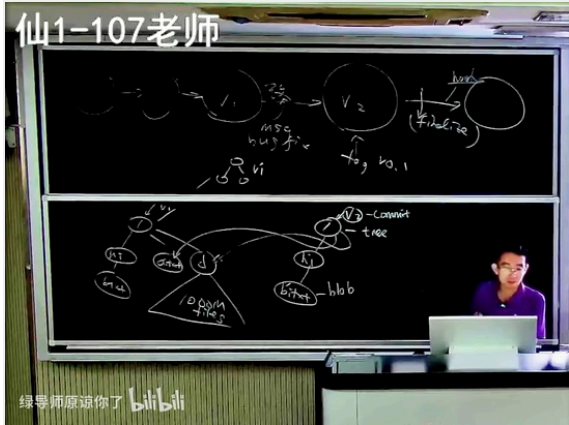
## 更多 best practice：回去读文档，让 AI 帮你学

- 讲者说这里还有很多很多 best practice。
- 比如可以在网上找到各种 commit 规范，有很多版本。
- 这些其实都来自工程实践经验。
- 他建议听众回去读这些文档，让 AI 帮你，就会学到更多。

## 为什么要及时 commit：复杂项目里改得越多，越可能出错

- 讲者提出“为什么要及时 commit”。
  - 理由：在复杂项目里，随着你修改的东西越多，你错的东西就越可能多。
  - 小项目刚开始的时候其实不要紧。
-

## 第 26 部分



### Best Practices: 让错误可恢复

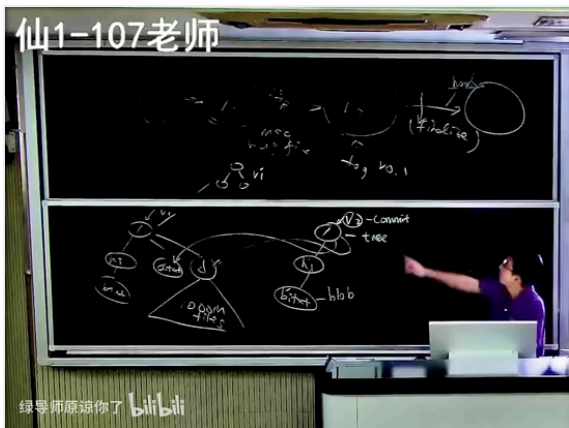
#### 一、给自己留退路

还有随处可见的 best practice。

- ◆ **及时 commit**: 实验前留检查点: 只在工作区里的修改, Git 未必能救。
- ◆ **备份与恢复**: 误操作后先保护现场, 再查 reflog: 远端 clone 不包含未提交文件。

#### 二、把探索整理成别人能接手的变化

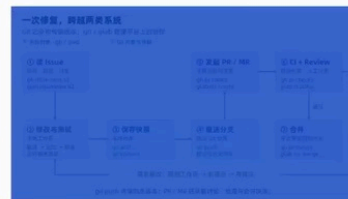
- ◆ **发布前整理**: 按一个意图拆分提交, 把修复和相应测试放在一起: 顺手格式化另开提交。
- ◆ **让历史可检验**: 整理后的提交能构建、能测试、review、revert、bisect 才有可靠的单位。



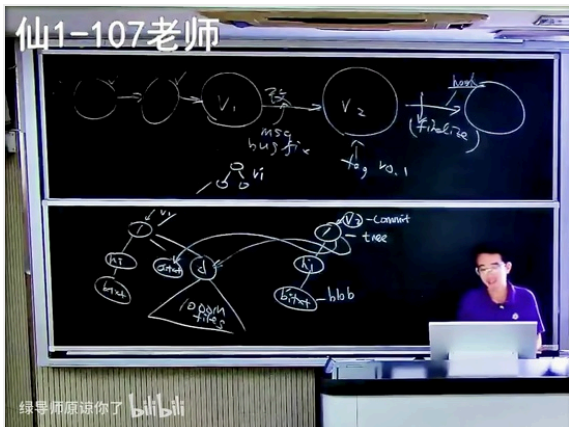
### Git 之外: gh 与 glab

#### 从问题到合入, 把流程接起来

甚至为了管理 non-git 的部分, 还专门有 gh, glab 这些工具。



- ◆ **Git 保存和传输版本**: gh / glab 操作平台上的 issue、PR/MR 与 CI 状态, 让人和 Agent 都能从命令行完成协作。



## 为什么必须及时 commit: 复杂项目会把大提交的 bug 定位成本放大

讲者给出的起点是: 在复杂项目里, 你修改的东西越多, 出错的可能性就越多。小项目刚开始其实不要紧, 一次提交两三千行代码都 OK; 但项目里一旦有错综复杂的关系, 一个大的 commit 可能 break 多处东西, 引入多个 bug, 后面又要引入多个 fix 来修它。

逻辑链：

1. 复杂项目 + 大 commit → 一次改动同时影响多处。
2. 多处被 break → 引入多个 bug。
3. 多个 bug 需要多个 fix，但这些 fix 都被归因到那个大 commit。
4. 如果大 commit 是 V2，后面修了 100 个 bug，都说“这是 V2 产生的”，就很糟糕：修不过来，也不知道 V2 到底修好没有。

## 小 commit 的好处：冤有头债有主，bug 和 fix 可以配对

---

讲者认为，如果每次 commit 都很小，只做一个修改，那“冤有头债有主”：这个 bug 就是你写出来的，你修完就完了；bug 所在 commit 和 fix commit 可以配对。这带来：

- **审计能力**：每个改动和修复有清晰的对应关系。
- **远端备份**：如果你改了很多，电脑又物理损坏，还没推到远端，就会丢失；做一个小提交后立即 **push**，就能得到互联网上的备份，电脑马上坏也不怕。
- **AI 可用能力**：讲者说 AI 很清楚可以 **review**、可以 **revert**、可以做 **bisect**（二分查找）。后面可能还会讲 **hooks** 和 **CI**。

## agent 在 GitHub / GitLab 上的行为：调 gh / glab，失败后回退到“古法编程”

---

讲者提到，如果项目托管在 GitHub / GitLab 上，会看到 AI agent 试图调用一些 **gh** 或 **glab** 命令；然后会看到一个 **command not found**，于是它又回退到“古法编程”，开始写 **git** 命令。

讲者给出的理由：GitHub 太 popular、太好用，所以值得为它做一个命令行工具。

## GitHub 的 issue / 评论不随 Git 持久化，因此需要额外命令行工具

---

讲者说明：Git 管理了很多代码快照，但 GitHub 上还有 **issue**，也可以对代码做 **comments**；这些东西不随着 Git 持久化。若想管理它们，就可以做一个命令行工具：

- **fetch** 一个 issue；

- 找 assign 给自己的 issue（需要登录）；
- 找到后还可以把 issue close 掉。

对 AI/agent 来说，这个命令行工具很好用：装了 gh 以后，可以在 agent 里设置一个 goal，让它完成一个很长的任务，并要求它做小步提交、可审计的提交、遵守分支规则、保持线性历史等。

## 讲者项目里的例子：大量小提交，很多他都不管

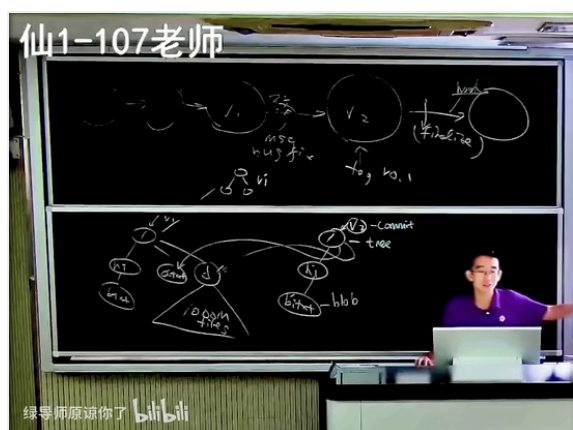
讲者举例：他的项目里就有大量这样的提交；从开始提交代码起，就有大量这种提交。他说自己很多时候都不管它们了。

## 今天最该记住的总结：Git 是软件工程，用 atomic change 推进快照

讲者强调，讲了这么多 Git，这门课讲的还是软件工程。最后的总结更重要：Git 本质上是一个软件，它要求我们把项目组织成用 **atomic change** 推进的快照。这是今天要记住的事情，所有东西都从这句话推出来：你组织一个软件项目，要管理很多快照，这样就可以随时开启一个新的分支。

小结：核心是把项目组织成由 atomic change 推进的快照。

## 第 27 部分



### 回到软件工程

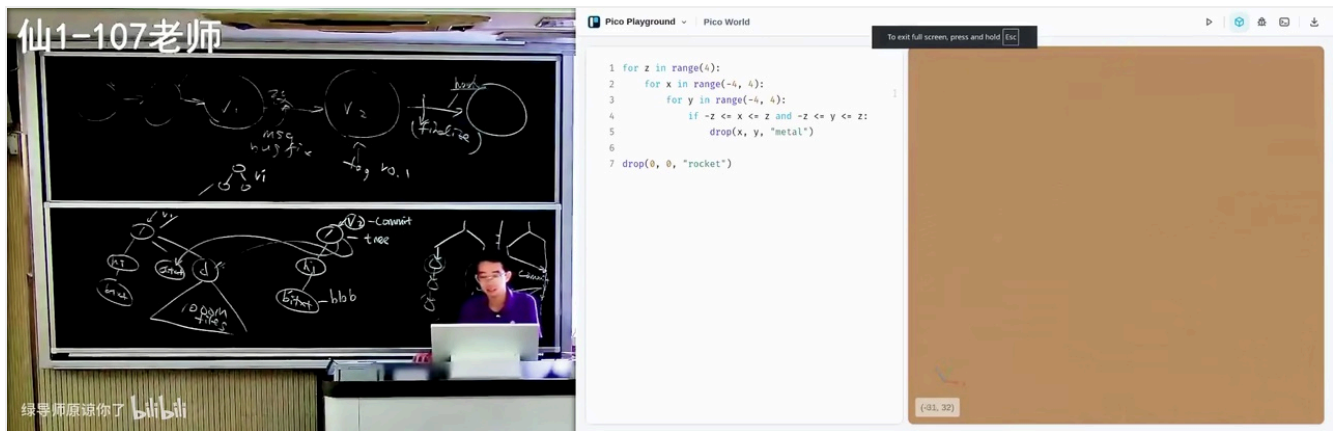
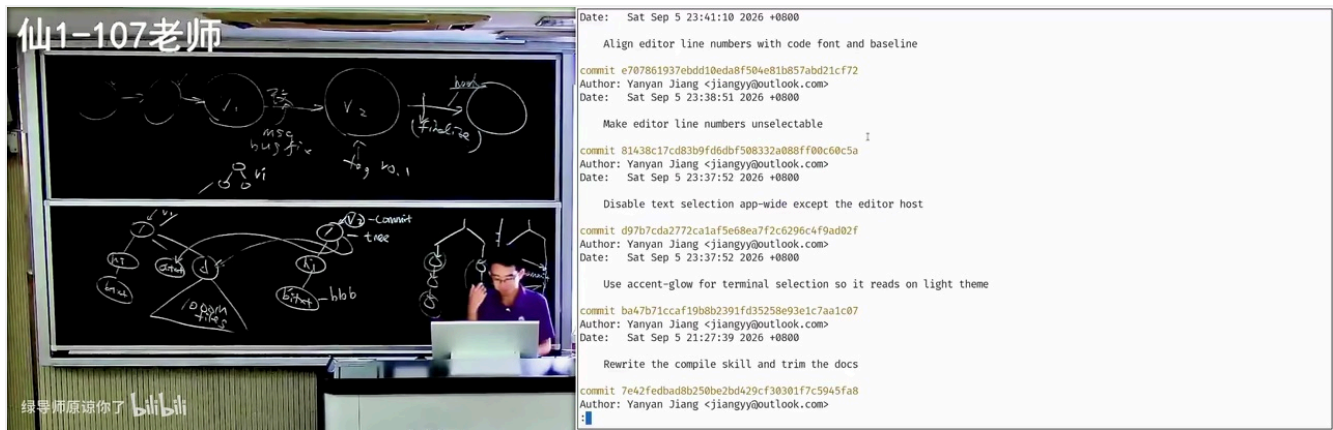
#### 项目 = 由 Atomic Change 推进的快照

可以分支，可以合并。这是人类成功的经验。

- 人无法处理“一团混沌”
- 必须分解成可以理解、可以审计的小任务
  - 出了问题可以 bisect，发生冲突可以解决
  - 有些项目要求 linear history：历史易于追溯，cherry-pick / blame 友好

#### AI-native 也要避免“一团混沌”

- 在无限大能力模型到来之前，这条法则依然成立
- ✦ 一个 change 对应一个意图：出了问题，才能单独解释、验证、撤销。



## 一切从“项目要管理很多快照”推出：分支、丢弃、合并、并行

讲者说，后面所有做法都从这句话推出：组织一个软件项目，本质上要管理很多快照。由此得到一条明确逻辑链：

1. 管理很多快照 → 可随时开启新分支做尝试。
2. 尝试失败 → 可以安全扔掉。
3. 尝试成功 → 可以合并进来。
4. 因此甚至可以同时推进多个主线，最后以合并完成，而不会互相干扰。

讲者用大作业经验说明：如果你要改好几个地方，改着改着很可能改乱，和这个部分、那个部分冲突。\*\*（整理者串联）\*\*分支/合并的价值就是把“多处同时改”隔离成可丢弃、可合并的尝试。

## 项目管理的本质：人类处理不了一团混沌，所以必须拆成小任务

---

讲者断言：项目管理的本质是人类没有办法处理一团混沌——很多需求、很多代码依次做一大波。于是很自然要：

- 把需求分解成可理解、可审计的小任务；
- 一小步一小步来。

小步之后，出了问题才能：

- 用 **bisect** 定位；
- 协作时用 **blame** 归因；
- 让冲突更容易解决。

讲者进一步举例：如果我在一个公共祖先上做了好多事，但每个提交都很小，那么发生冲突时，我可能知道是哪一个小提交冲突；剩下不冲突的部分合并起来会比较容易。\*\*（整理者串联）\*\*这里的关键不是“小步”本身，而是小步把定位、归因、合并都变成局部问题。

## 线性历史：AI 能轻松 rebase/resolve，所以更值得强制

---

讲者区分两种做法：

- 普通 **merge**：把一个提交合上来；一个 commit 可以有**两个 parent**；历史继续分叉、合并，最后形成很典型的 Git 结构。
- 现在更流行 **linear history**：讲者说反正都是 AI，**resolve**、**rebase** 都很容易，所以他会强行要求整个 commit history 都是线性的，“好像是线性的一样”。

讲者也说这不全是好处，但至少对现在维护项目来说：

- 历史更易于追溯；
- 对 **cherry-pick** 和 **blame** 更友好。

讲者的时间限定是：在无限大能力的模型到来之前，这件事都是好的。链条是：把 Git 记录拆分得好 → 很容易找到这次修改由谁引入 → AI 维护项目效率更高 → 省 token；有些文件就可以不用去读。

# 架构阶段先用单线程 + 高智力模型，把组件库接口定好

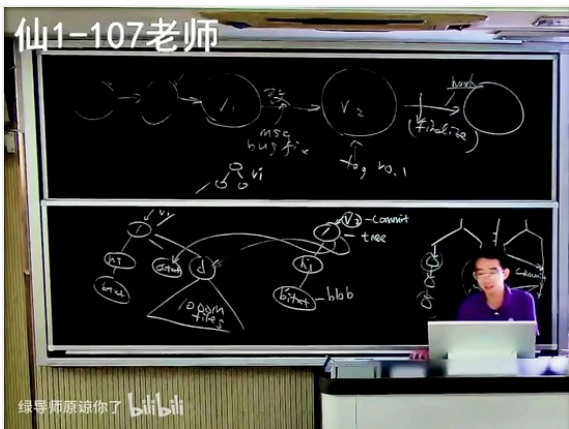
讲者分享自己的项目管理经验：虽然知道要小步前进，但具体怎么做？他会在架构设计阶段先用单线程，用一个智力比较高的模型跟它讨论。

讲者强调这很重要，因为：如果软件设计不好，后面就是无穷迭代。他提到第一次课展示过这个例子：他会把组件库的接口定得非常好。接着他准备展示一个例子，说明他为什么信任 AI 的代码：项目里有很多互相关联的组件，程序会运行、会产生结果。

**\*\*（整理者串联）\*\***因此，讲者的思路是：先固定架构和接口，减少后续无穷迭代；接口稳定后，才更容易信任 AI 写出的实现与维护代码。

## 第 28 部分

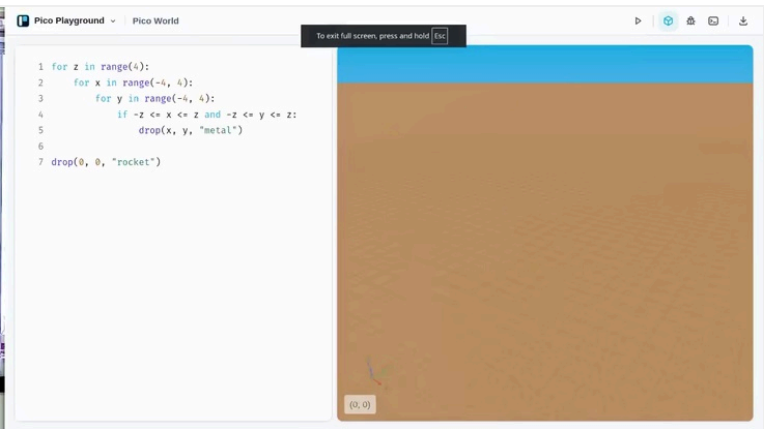
仙1-107老师



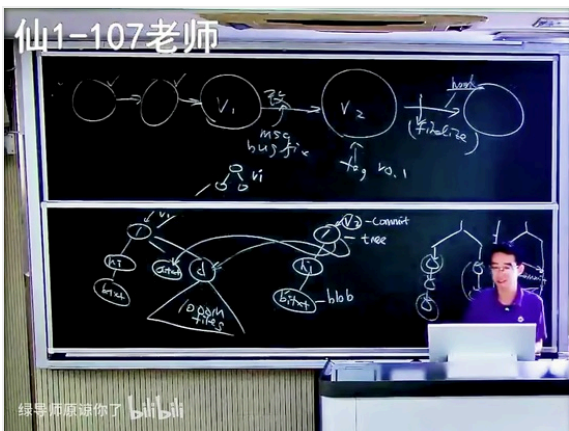
绿导师原谅你了 bilibili

Pico Playground - Pico World

```
1 for z in range(4):
2   for x in range(-4, 4):
3     for y in range(-4, 4):
4       if -z <= x <= z and -z <= y <= z:
5         drop(x, y, "metal")
6
7 drop(0, 0, "rocket")
```



仙1-107老师



绿导师原谅你了 bilibili

### Agent 能力之内的项目

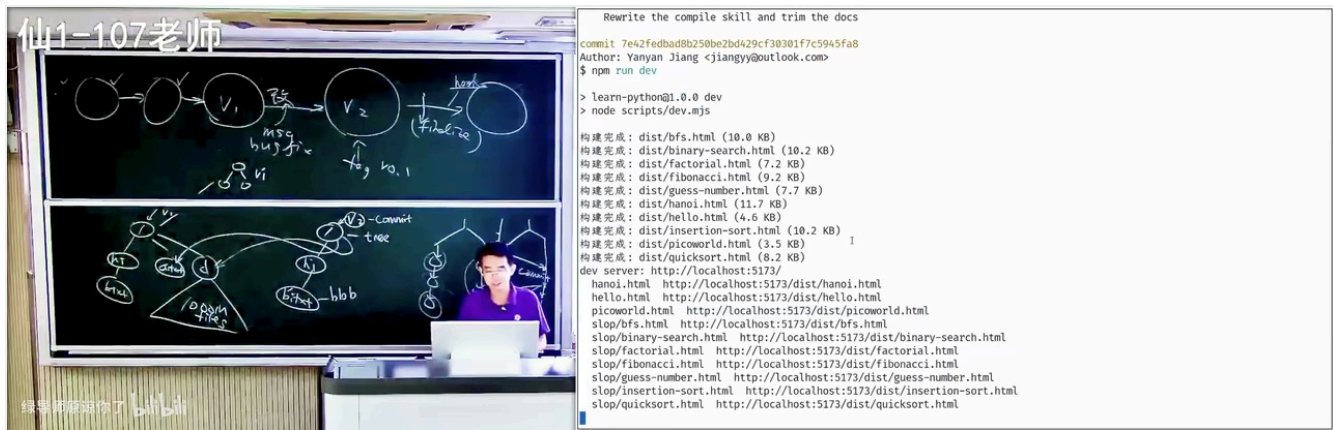
**架构设计：单线程，反复讨论**

比如 Pico Playground：和高智力模型讨论，直到收敛。

- 将设计落到文档、规则、示例：比如测试要做到什么程度
- 仍然要学操作系统、编译原理、数据库：这里有人类积累的好设计
- ♦ 架构先确定接口与不变量，后续任务才有可以独立行动的边界。

**实现：有限并发，立即反馈**

- $n \leq 3$  个 terminal
- AGENTS.md 明确协作规则，例如 linear history
- 人脑并发控制：布置不冲突的任务
- 不管 AI 的质量如何，都立即检查、视觉验收、反馈



## 为什么讲者信任 AI 代码：架构把接口和共享状态压到极小

讲者解释自己为什么信任 AI 写的代码。这个项目里有很多互相关联的组件：程序运行会产生结果；拖动进度条不仅关联到程序编辑器，还关联到上方/其他可视化区域。讲者的处理方式是：

1. 把绝大部分接口设成 **functional de**（原词；随后讲者解释为 **pure function**）。连 **trace** 也是程序的一个 **pure function**。
2. 有一个标准的程序 **DERVISION graph**（原词如此；疑似 **derivation graph**，未确认），要求 **agent** 按这种方式展开。
3. 所有全局互相影响的状态都注册到一个全局地方。比如光标位置怎么影响可视化，包括菜单条，都收进这个非常干净、非常小的状态里。
4. 因此这个 **playground** 的核心共享状态只有十几行，组装也只有十几行，剩下所有东西都是黑盒子。
5. 讲者直接要求 **agent** 在设计 **visualization** 时禁止看任何库函数的实现，只看接口。

结果是花费很省：上下几万个 token 就能做出一个这样的 playground。

## 多 agent 协作：不调视觉模型/浏览器，人放另一块屏实时验收

架构设计完成后，讲者会开三个（或几个）terminal/agent。他的 **agent.md**（原词：agent点MD）里有明确协作规则，讲者说可能后来改过一些内容。

规则和做法包括：

- 不需要调用视觉模型去检查；讲者说 AI 很喜欢做这件事，会花很多很多 token。

- 也不调用浏览器任务。
- 讲者在旁边另一块屏放一个浏览器，浏览器实时刷新，他肉眼观察。
- 他开始开三个 agent，哪里不满意就骂他；肉眼看着右边那个东西不停地变好，最后问题解决。
- 讲者判断：这是一个 agent 能力范围之内的项目。

## agent 能力继续增长：更多项目会被“轻量讨论 + 剩下完成”，但版本控制会成为问题

---

- 讲者非常确信：agent 能力会不断增长。
  - 因此越来越多项目能被 agent 以这种方式完成：先比较轻量地讨论，然后剩下部分由 agent 完成。
  - 讲者认为这是一件非常可怕的事情。
  - 他留下问题：如果 agent 快了 1000 倍，我们还怎么管理版本控制？下一节课再讨论。
-